CROSSING INTO THE FUTURE OF SIGNATURES WITH GO

A PRACTICAL GO IMPLEMENTATION OF THE CROSS SIGNATURE SCHEME

KRISTIAN HØI LIBORIUSSEN, 202006178
RASMUS ØSTERGAARD, 202004625

CRYPTOGRAPHY AND CYBER SECURITY

MASTER'S THESIS

June 2025

Advisor: Diego F. Aranha

AARHUS UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE

CROSSING INTO THE FUTURE OF SIGNATURES WITH GO

A practical Go implementation of the CROSS signature scheme

KRISTIAN & RASMUS

Master's Thesis

Department of Computer Science
Faculty of Natural Sciences
Aarhus University

June 2025

ABSTRACT

This thesis focuses on the Post-Quantum Signature Scheme "CROSS" that is currently part of the NIST standardization process for additional signature schemes. CROSS will be investigated theoretically and practically through a re-implementation of the scheme in Go. The optimizations used in the scheme will be explained both from a theoretical and an implementation point of view. Our thesis in general provides a more approachable introduction to the CROSS scheme and the general field of post-quantum code-based signature schemes.

Our implementation works for all defined versions and security levels of CROSS and passes Known Answer Tests (KATs) made by the reference code to ensure signature compatibility. The implementation is functional according to NIST requirements and KAT tests, while being reasonably secure against timing attacks. However, further optimizations can be performed to make it more suitable for some practical applications.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LISTINGS

Part I

BACKGROUND

This part presents the Post-Quantum Digital Signature Scheme *CROSS* and discusses its context within the NIST call for additional signature schemes. We introduce the Syndrome Decoding Problem (SDP), a classical problem for code-based cryptography. We also introduce the Restricted Syndrome Decoding Problem - R-SDP and R-SDP(G) , two variants of SDP used in CROSS. Finally, we show fundamental concepts for building Post-Quantum schemes such as hash functions, Merkle trees, and seed trees.

1 INTRODUCTION

1.1 MOTIVATION

In this thesis, we aim to investigate and implement the Post-Quantum digital signature scheme "Codes and Restricted Objects Signature Scheme" (CROSS) which has been proposed to NIST's "Post-Quantum Cryptography: Additional Digital Signature Schemes" call. According to the NIST requirements, the implementations delivered for the call are made in C, so we will implement the scheme in a newer language; Go, a more type-safe language built for large-scale applications and network communication [21]. This allows the scheme to be used in different types of systems, such as web applications and large scale distributed application [21].

Among the 40 candidates submitted to NIST, we chose to implement a code-based scheme because code-based schemes are a fairly self-contained area. One of our group members also had previous experience with older encryption schemes based on error-correcting codes from 2G and 3G networks. Initially, we considered MEDS or Wave, two candidates from NIST's first round. However, both were removed shortly before our project began, following the announcement of the Round 2 candidates, which led us to select CROSS.

1.2 GOALS AND STRUCTURE

Our Go implementation should be compatible with signatures from the C implementation and only utilize secure and trusted libraries in order to guarantee its security. We will measure performance of the implementation compared to the C implementation, but the main goal of the project is not to outperform C; instead, we aim to provide an easy to import and use Post-Quantum signature scheme. We will also try to ensure that our code is not vulnerable to timing attacks, both with a theoretical analysis and a practical estimation tool. Doing our own implementation also gives us the opportunity to read the CROSS specification with a different focus rather than a purely theoretical analysis of the scheme; instead, we will also have to focus on the implementation specific details in order to achieve an exact compatibility with the reference code.

Our thesis will introduce CROSS and the parts used for implementing

it, this will include common constructions and optimizations used for designing and implementing Post-Quantum Cryptography (PQC). We will introduce the protocol and our implementation, providing pseudo-code adapted to our implementation, as well as a walk-through of the code details. Finally, we will evaluate the goals mentioned and compare the advantages and disadvantages between the Go and C implementations.

1.3 NIST

The National Institute of Standards and Technology (NIST) is a US federal agency responsible for developing and promoting standards, including cryptographic standards used across industries and governments. In cryptography, NIST plays an important role in evaluating, selecting, and standardizing secure algorithms for encryption, digital signatures, and key exchange. Each submission needs to be able to run with three different security levels, these security correspond to the difficulty of breaking AES keys. Breaking a scheme with security level 1 is equivalent to breaking AES keys of 128 bits. For security level 3 the difficulty is equal to breaking keys of 196 bits, and finally for security level 5 this is equivalent to breaking AES keys of 256 bits.

To determine which cryptographic algorithms should become official standards, NIST conducts open calls for submissions. In these calls, researchers and cryptographers from all over the world can submit their proposed schemes for consideration. The submitted algorithms are then subjected to public review and thorough testing in various settings. This process ensures transparency and helps to identify potential vulnerabilities or performance issues prior to standardization [29].
Notable outcomes of NIST's standardization efforts include:

- **Rivest–Shamir–Adleman (RSA)**, a widely used public-key cryptographic algorithm that provides both encryption and digital signatures. RSA was approved for digital signatures by NIST in 1998 as part of the Digital Signature Standard (DSS) and is based on the mathematical difficulty of factoring large integers. It has become one of the most widely adopted digital signature schemes in both government and industry applications [34].
- **Advanced Encryption Standard (AES)**, which was officially adopted in November 2001 following a public competition organized by NIST. AES replaced the older Data Encryption Standard (DES) and became the new standard for symmetric encryption [28]. AES was the first encryption standard selected through a public competition.

The "Post-Quantum Cryptography: Additional Digital Signature Schemes" call was made to focus on signature schemes using a different security assumption than structured lattices [33], since two of the previously standardized schemes: CRYSTALS-Dilithium and FALCON are lattice-based, while the third, SPHINCS+, is hash-based [30]. As of June 2025, CROSS is one of the 14 candidates still left in Round 2 in this call for new standard signature schemes [35].

1.4 CROSS

CROSS is a code-based digital signature scheme, built upon the Restricted Syndrome Decoding Problem (R-SDP) [5]. It is one of two code-based candidates selected for Round 2 of NIST's post-quantum digital signature call, alongside LESS. A variant of CROSS targets the R-SDP(G) problem, which constrains solutions to a specific subgroup. This modification enables smaller signature sizes and improved performance. However, both R-SDP and R-SDP(G) are relatively recent proposals in the cryptographic literature, with the first paper on the topic published in 2020 and revised in 2021 [3]. Further research is required to establish confidence in their security.

CROSS offers three different settings for each of the two problems: Balanced, fast, and small. The balanced and small variants both follow the same signature generation structure, with the small variant achieving a smaller signature size at the cost of increased computation time. Fast, on the other hand, offers faster operations at the cost of a larger signature size by using a different tree structure [5]. This flexibility makes CROSS suitable for a variety of systems, such as where a small signature size might be needed due to limited data transfer speed or where fast operation is necessary on an embedded device.

In Figure 1.1, the self-reported cycle counts for round 1 submissions are shown. The arrows are related to how much the schemes have moved when removing the cycle count for signing. This was shown in a previous figure in a NIST presentation [39]. All the measurements for this figure were done using the cycle count for security level 3.

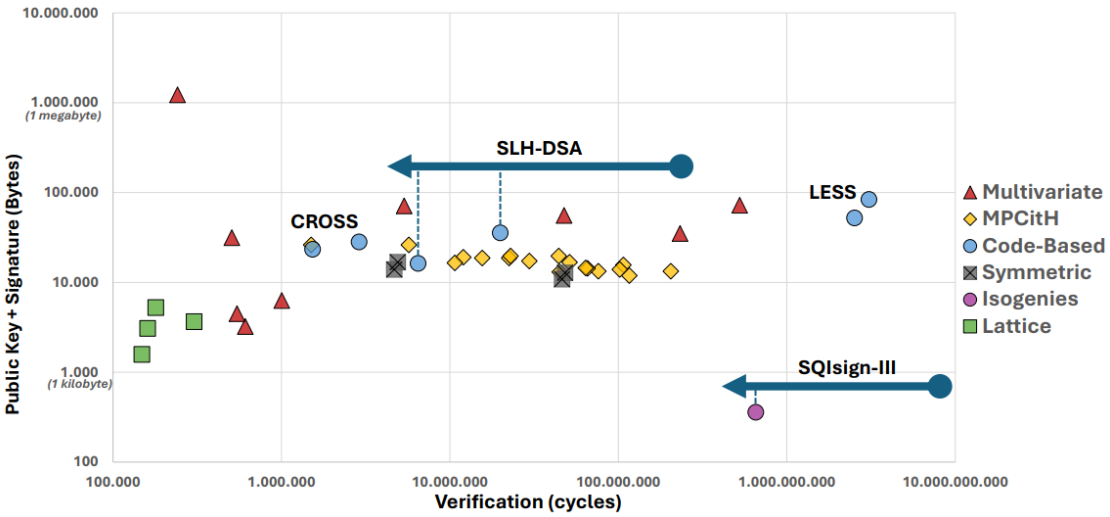

Figure 1.1: Performance summary of public key and signature size along with number of cycles needed for verification in log scale [38]

As in the comparison conducted by NIST and PKIC in January 2025 [38], CROSS demonstrates favorable performance in terms of verification speed and combined size of the public key and signature size. Although lattice-based schemes generally outperform CROSS on both metrics, this specific NIST call aims to standardize a non-lattice-based alternative.

1.5 OUR GENERATIVE AI DECLARATION

We have utilized ChatGPT 4o as a help with proofreading, for grammar, sentence structure, improved choice of words, and proper tone. We provided the model with the following prompt:

> "I wish you to critique the following Master's Thesis abstract/paragraph/etc. as a seasoned Computer Science professor and thesis supervisor would. You are not to offer a corrected version of the text, but you are to point out strengths and weaknesses, so that the recipient can correct the text themselves."

This prompt was recommended by Niels Olof Bouvin's template for using AI in thesis work [11].

We utilized ChatGPT during our programming of CROSS as an aid to debug and come up with quick suggestions for potential causes to investigate further, mainly for generating the test vectors in C and adjusting the CMake files. ChatGPT also provided suggestions for some LaTeX formatting bugs that we encountered during writing. Finally, we utilized the Overleaf implementation of Writefull's AI for proofreading.

2 PRELIMINARIES

2.1 NOTATION

The mathematical symbols used in the CROSS specification [5] are summarized in Table 2.1. In this work, we follow the notation and adopt the specification's mathematical conventions, with only one deviation: we use $\|$ instead of $|$ to denote concatenation. The conventions are as follows:

- vectors over $\mathbb{F}_p$ are in bold letters, e.g., $\mathbf{e} \in \mathbb{E}^n \subset \mathbb{F}_p^n$
- matrices over $\mathbb{F}_p$ are in bold capital letters, e.g., $\mathbf{H} \in \mathbb{F}_p^{(n-k)\times n}$
- vectors over $\mathbb{F}_z$ are in bold overlined letters, e.g., $\overline{\mathbf{e}} \in \mathbb{F}_z^n$ and $\overline{\mathbf{e}}_G \in \mathbb{F}_z^m$
- matrices over $\mathbb{F}_z$ are in bold overlined capital letters, e.g., $\overline{\mathbf{M}} \in \mathbb{F}_z^{m\times n}$
- concatenation between x, y is denoted as $x\|y$
- in algorithms we write $a \leftarrow b$ to denote that a is assigned the value b and $a \xleftarrow{\$} A$ denotes that a is drawn uniformly at random from A.

The primary cryptographic notation used throughout this work is summarized in Table 2.2.

2.2 R-SDP & R-SDP(G)

2.2.1 Syndrome Decoding

The Syndrome Decoding Problem (SDP) is a classical problem in coding theory, and is known to be NP-complete [8]. SDP has been used as a basis for multiple cryptographic systems, some of which have been part of previous NIST calls for Post-Quantum algorithms [2].
SDP can be expressed by the equation: $\mathbf{e}\mathbf{H}^\top = \mathbf{s}$, where, $\mathbf{e} \in \mathbb{F}_p^n$ is the error vector, this is the solution we want to find. The parity-check matrix $\mathbf{H} \in \mathbb{F}_p^{(n-k)\times n}$, represents a system of linear equations. $\mathbf{s} \in \mathbb{F}_p^{n-k}$ is the syndrome vector, computed as the product of $\mathbf{e}$ and the transpose of $\mathbf{H}$. The problem is to find an error vector $\mathbf{e}$ given both $\mathbf{H}$ and $\mathbf{s}$.

Symbol	Meaning
p, z	Prime numbers, $z < p$
$\mathbb{F}_p$	Finite field with p elements
$\mathbb{F}_p^*$	Multiplicative group $\mathbb{F}_p \setminus \{0\}$
$\mathbb{F}_z$	Finite field with z elements
$\mathbb{E}$	Cyclic subgroup of $(\mathbb{F}_p^*, \cdot)$, with generator g of order z
$\star$	Component-wise multiplication
G	Subgroup of $(\mathbb{E}^n, \star)$ of size z^m
Id_ℓ	Identity matrix of size $\ell \times \ell$
n	Code length and length of restricted vectors
m	Size of the subgroup G is z^m, $m < n$
k	Code dimension, with $k < n$
λ	Security parameter
t	Number of rounds
w	Weight of the second challenge
$\mathcal{B}_{(t,w)}$	Hamming sphere of vectors in $\mathbb{F}_2^t$ with radius w
$HW(t)$	Hamming weight of the binary representation of t

Table 2.1: Mathematical symbols

An important constraint in SDP is the weight w of the error vector $\mathbf{e}$. The weight refers to the Hamming weight, which describes the number of non-zero entries in a vector [10]. This constraint has a significant impact on the problem's hardness: increasing the weight w increases the size of the search space and the number of non-zero variables, which makes the problem harder. However, if w is too large, the problem may allow multiple solutions, potentially making it easier to find a valid $\mathbf{e}$ [4].

2.2.2 Restrictions on Syndromes

Restricted Syndrome Decoding Problem (R-SDP) is a variant of the classical Syndrome Decoding Problem (SDP), where the error vector $\mathbf{e}$ is restricted to be in a multiplicative subgroup $\mathbb{E}^n$ of the finite field $\mathbb{F}_p^n$. The further restricted version, R-SDP(G) constrains $\mathbf{e}$ to a subgroup G of $\mathbb{E}^n$ [9]. More formally, let $\mathbb{F}_p$ be a finite field with p elements, and $g \in \mathbb{F}_p^*$ be a generator of a cyclic subgroup of prime order z. The restricted vector's entries belong to the subgroup $\mathbb{E} = \langle g \rangle = \{g^i \in \{1, ..., z\}\} \subseteq \mathbb{F}_p^*$. The goal of R-SDP is to find a vector $\mathbf{e} \in \mathbb{E}^n$ such that $\mathbf{s} = \mathbf{e}\mathbf{H}^T$, given a syndrome $\mathbf{s} \in \mathbb{F}_p^{n-k}$ a parity-check matrix $\mathbf{H} \in \mathbb{F}_p^{(n-k)\times n}$, and a restricted set $\mathbb{E}$. The motivation for using restricted vectors and element-wise multiplication is that it is isomorphic to vectors in $\mathbb{F}_z^n$ with addition, $(\mathbb{E}^n, \star) \cong (\mathbb{F}_z^n, +)$ [5].

Symbol	Meaning
`Msg`	Message to be signed
`sk`	Secret key
`pk`	Public key
`Sgn`	Signature
`Hash`	A cryptographic hash function with codomain $\{0,1\}^{2\lambda}$
`Salt`	Binary string of length 2λ randomly drawn for each signature generation
$\texttt{Seed}_x$	Seed used to draw x
$\texttt{digest}_x$	Digest of a cryptographic hash function on x
$\texttt{cmt}_0[i]$, $\texttt{cmt}_1[i]$	Commitments for round i
$\texttt{chall}_1$	First challenge vector in $(\mathbb{F}_p^\star)^t$
$\texttt{chall}_2$	Second challenge vector in $\mathcal{B}_{(t,w)}$
`resp`$[i]$	Response to the second challenge for round i
$\mathcal{T}$	A tree structure where each node consists of a λ- or 2λ bit-string depending on its context
$\mathcal{T}'$	A reference tree structure where each node consists of a single bit
`CSPRNG-`$S(\cdot)$	A cryptographically secure pseudo-random number generator with output in the set S

Table 2.2: Notation employed for inputs, outputs and cryptographic components

In R-SDP(G) solutions are in the subgroup $(G,\star) \leqslant (\mathbb{E}^n,\star)$, where $G = \langle a_1,...,a_m\rangle = \{a_1^{\overline{u}_1} \star ... \star a_m^{\overline{u}_m} | \overline{u}_i \in \mathbb{F}_z\}$ with $|G| < |\mathbb{E}^n| = z^n$.
CROSS utilizes linear transitive maps:

- R-SDP, maps $\overline{v} : \mathbb{E} \to \mathbb{E}$
- R-SDP(G) , maps $\overline{v}_G : G \to G$

This can be done since both $\mathbb{E}$ and G are closed under multiplication, which allows us to compute $\overline{v}(\mathbf{e}) = \mathbf{e} \star \mathbf{e}', \mathbf{e}' \in \mathbb{E}^n$.
The subgroup $\mathbb{E} \leqslant \mathbb{F}_p^*$ is generated using the public parameter $g \in \mathbb{F}_p^*$ and is set to $g = 2$ for R-SDP and $g = 16$ for R-SDP(G) [5].

2.2.3 Solving SDP, R-SDP, and R-SDP(G)

Among the algorithms for solving SDP, generic Information Set Decoding (ISD) algorithms are some of the most prominent. The efficiency of ISD algorithms is heavily based on the Hamming weight constraint, as these algorithms essentially operate by searching for zero entries in the error vector $\mathbf{e}$. However, solving large SDP instances using ISD remains computationally challenging [9]. There is a website dedicated

to Syndrome Decoding Problems, where researchers submit their solutions, the algorithm used, and the time it takes to solve them. The problems are defined in terms of the length of $\mathbf{e}$ and the Hamming weight. The largest problem currently solved is with an error vector of length $|\mathbf{e}| = 570$ and Hamming weight $w = 70$, which took 45.32 days [18] using a variant of the MMT algorithm, which itself is an ISD algorithm [26].

It appears that these generic decoders perform worse on both R-SDP and R-SDP(G) as the restriction on the error vector $\mathbf{e}$ allows an increased weight w while maintaining a low probability of having multiple solutions [4]. Therefore, despite the use of both R-SDP and R-SDP(G) in cryptographic protocols being quite new, they might give better performance compared to SDP, since the Hamming weight can be increased without having a high probability of multiple solutions existing. This enables smaller matrices and vectors than would have been needed if SDP had been used as the base problem. Although efficient attacks have not been found, an attacker could potentially exploit the structure of G in R-SDP(G) to perform some algebraic attack on R-SDP and R-SDP(G) [9].

2.3 SHAKE

The CROSS signature scheme relies on two auxiliary cryptographic primitives: a cryptographically secure pseudorandom number generator (CSPRNG) and a hash function. Both are instantiated using the same SHAKE variant, but with different domain separator constants. Domain separation ensures that each use of the hash function behaves like an independent random oracle, satisfying the assumptions required by the Random Oracle Model (ROM). [6].

In the Random Oracle Model, we assume that the hash function essentially works as a public black box which, when given an input, always returns the same random output for that input [15]. Since the outputs are random, it should be impossible to reverse them, besides using a brute-force approach of simply iterating over inputs until a desired output is found. In practice, hash functions are designed to be computationally indistinguishable from random functions under the assumed security levels. This means that inverting these functions and finding collisions should be infeasible within those security bounds [15]. If the same domain separator is used for two different purposes, then the hash function would return the same output on the same input in both cases. This is problematic since the ROM assumes that the output is truly random and an output would not be truly random if two "different" oracles return the same response. Using domain

separators is a technique that prevents this issue and allows us to turn a random oracle into many independent random oracles [7].

The specific SHAKE variant used depends on the NIST security category of the CROSS instance. Table 2.3 shows the choice of SHAKE variants for each security level.

NIST category	**CSPRNG**	Hash
1	SHAKE128	SHAKE128 with 256-bit output
3	SHAKE256	SHAKE256 with 384-bit output
5	SHAKE256	SHAKE256 with 512-bit output

Table 2.3: SHAKE variants used in CROSS, for each NIST category

These levels are chosen due to the security levels of the two different SHAKE variants, this is shown in the NIST's guidelines on using SHAKE from the SHA-3 standard [27].

Function	**Output Size**	**Collision**	**Preimage**	**2nd Preimage**
SHAKE128	d	$\min(d/2, 128)$	$\geqslant \min(d, 128)$	$\min(d, 128)$
SHAKE256	d	$\min(d/2, 256)$	$\geqslant \min(d, 256)$	$\min(d, 256)$

Table 2.4: Security strengths of the selected hash functions [27]

As shown in Table 2.4, the collision resistance of each SHAKE variant is equal to the minimum of either half of the output length or the security level of SHAKE (128 bits for SHAKE128, and 256 bits for SHAKE256). These properties ensure that SHAKE128 and SHAKE256 provide exactly the level of security required for the NIST security categories, as long as they are used with the appropriate output length, which is shown in Table 2.3.

SHAKE is based on a sponge construction, where input is absorbed into the state through a series of steps. The output is then retrieved from the state through another series of steps. For SHAKE, this is done by initializing an internal state known as the sponge. First, the input is padded, then the padded input is split into r length blocks and an XOR operation is performed on part of the internal state and these blocks, one at a time, updating the internal state. Between each block absorption, a function f is applied to the internal state of the sponge and these steps are repeated until every block has been absorbed.

To support arbitrary length outputs, part of the internal state equal to r bits can be extracted, if more output is needed, the function f can be applied to the internal state. After the function f has been applied, part of the internal state has been updated and can then be extracted

again, and this can be repeated until the desired output length has been achieved. A certain part of the state is never extracted, which ensures that future sampling from the same state remains secure.

For this reason, SHAKE and other hash functions are variable time based on input length. This is important to keep in mind when we try to measure if our implementation is constant-time in the evaluation Section 5.4. Different message lengths would naturally make a timing difference that would not be related to the key or any difference in behavior, other than simply taking more time to process with SHAKE. Since the message is publicly known, this time variance does not reveal any secret information.

2.4 TREE ALGORITHMS

CROSS supports three versions: small, balanced, and fast. The tree structures vary depending on the version in use. For the small and balanced versions, trees compress the signature size by reducing the amount of data that needs to be transmitted. In contrast, the fast version does not benefit from tree-based compression due to the parameter choices. In the small and balanced versions, the trees are constructed as unbalanced binary trees composed of multiple complete binary subtrees, as illustrated in Figure 2.1. This design ensures that every node has a sibling, which facilitates efficient computation and verification. Both Merkle and seed trees share the same structure, though the precise size and shape depend on the parameters used for the specific instance.

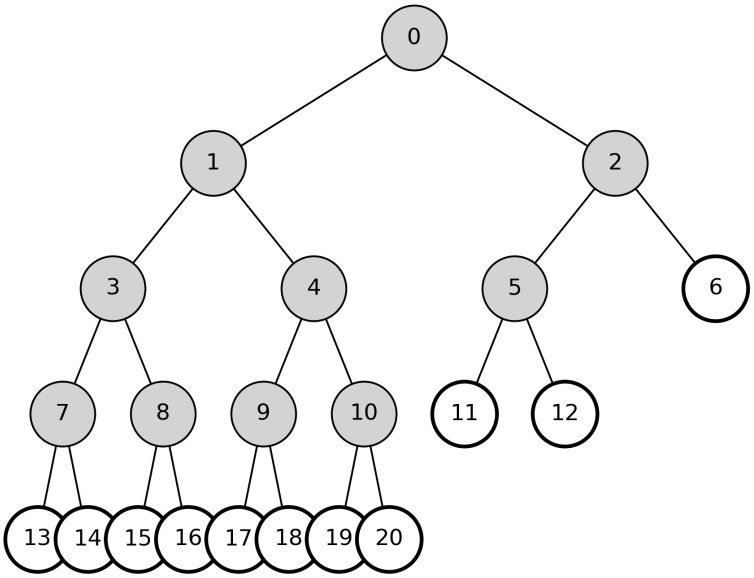

Figure 2.1: Example of a tree structure for balanced and small with t = 11

In the fast version, the tree consists of 3 layers: the root, an intermediate level, and a leaf level. The root node has four children that form the intermediate level. Each node in the intermediate level has a variable number of children, determined by the number needed to ensure the tree contains t leaf nodes, which are filled from left to right. An example of this tree structure is shown in Figure 2.2 below.

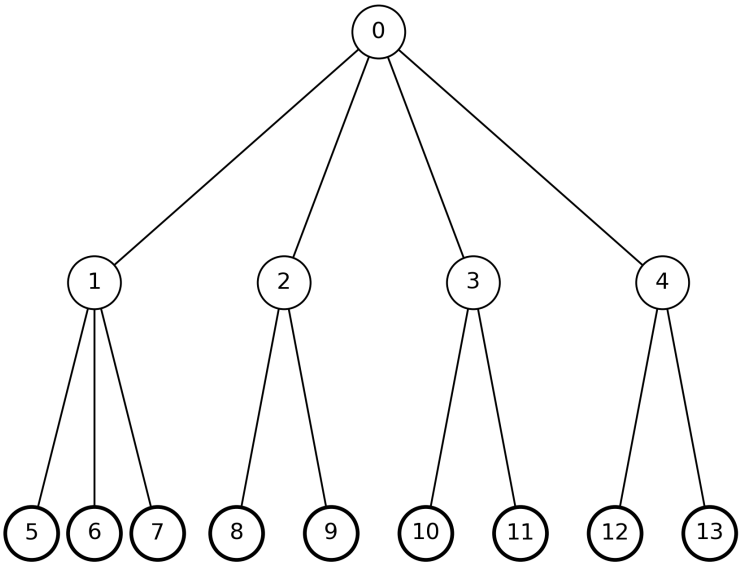

Figure 2.2: Example of a tree structure for fast with t = 9

2.4.1 Merkle Trees

The Merkle tree is constructed bottom-up from a list of t seeds (commitments), where each $\mathsf{cmt} \in \{0,1\}^{2\lambda}$. The structure is built by recursively computing the parent node through hashing the concatenation of its children's values. Each parent node is computed as $\mathsf{parent\_hash} = \mathsf{Hash}(\mathsf{cmt}_L \| \mathsf{cmt}_R)$, where $\mathsf{parent\_hash} \in \{0,1\}^{2\lambda}$. The resulting hash is assigned to the parent node. This process is then repeated until the root has been computed. The Merkle tree is later used by the verifier to recompute the Merkle root from a given leaf and its path.

Merkle trees are used for commitments, providing a root as the commitment to a set of values. Requirements for commitments are treated in the protocol chapter 3.1. Since the root is derived through repeated cryptographic hash operations over the tree's internal nodes, it is computationally infeasible to invert. To reveal some or all of the committed values, the prover simply supplies the relevant leaf values along with the minimal set of sibling nodes required to reconstruct the path to the root. This enables the verifier to independently recompute the root and check it against the original commitment. Due to the logarithmic depth of the Merkle tree, only $\mathcal{O}(\log_2 n)$ nodes needs to be transmitted to verify that the revealed commitments are valid [25].

The implementation of CROSS requires three operations on the Merkle trees: TreeRoot, TreeProof, and RecomputeRoot. The TreeRoot function takes as input the t commitments, places them on the leaf nodes, and recursively builds the tree until it reaches the root. The 2λ-bit string computed at the root is then returned.
The TreeProof function takes the commitments and a Boolean challenge vector chall_2 as input. The function constructs the Merkle tree and traverses it from the leaves to the root. The nodes included in the proof are determined by the challenge vector chall_2. If both children of a node would be included in the proof, they can instead be replaced by their parent node to reduce memory and computation when recomputing the root. When all the necessary nodes have been added to the proof and the size has been minimized, the proof is returned.
The RecomputeRoot function takes as input the recomputed_leaves, the Merkle proof proof, and the Boolean challenge vector chall_2. The tree is reconstructed using the recomputed leaves and the proof, and the recomputed root is returned. Implementation details are provided in Section 4.2.2.

2.4.2 Seed Trees

Seed trees, also known as GGM trees [20], are constructed top-down from a root seed, which is recursively expanded into t leaf nodes. This construction begins with a λ-bit seed at the root. Each node in the tree contains a seed $s \in \{0,1\}^\lambda$. To expand a parent node, a CSPRNG is used. The CSPRNG takes as input the concatenation $s \| salt \in \{0,1\}^{3\lambda}$, where $s \in \{0,1\}^\lambda$ is the parent seed and $salt \in \{0,1\}^{2\lambda}$ is a public salt value, which gets stored in the signature. The CSPRNG outputs a bit string $x \in \{0,1\}^{2\lambda}$, which is split into two child seeds:

$$x = x_L \| x_R, \quad \text{where } x_L, x_R \in \{0,1\}^\lambda$$

The values x_L and x_R are assigned to the left and right child nodes, respectively. This expansion process is repeated recursively, using a loop structure, until all t leaf nodes have been derived.

The seed tree functions are a puncturable pseudorandom function (PPRF) by providing a structured way to expand an initial random seed into an arbitrary number of pseudorandom values. This is done by using a cryptographically secure pseudo-random number generator (CSPRNG). Seed trees allow for deterministic generation of all node seeds in a tree from a single root seed.

In a tree setup, a PPRF is a type of pseudorandom function where even if some of the nodes in the tree have been published, it is still computationally infeasible to derive the seeds of the hidden nodes unless their parents are known [43]. This reduces the number of bits that need to be transmitted, since the receiver can derive multiple pseudorandom values from fewer seeds. This construction allows for a smaller signature size since only a path in the seed tree needs to be transmitted, rather than all seed leaves individually.

The implementation of CROSS requires three operations on the seed trees: `SeedLeaves`, `SeedPath`, and `RebuildLeaves`. The `SeedLeaves` function generates the seed tree from the root and returns t leaf nodes.
The `SeedPath` function also generates the tree but, in addition, takes as input a Boolean challenge vector $chall_2 \in \{0,1\}^t$ and outputs the subset of leaf nodes specified by the challenge vector.
The `RebuildLeaves` function takes the salt, the revealed seed path Path, and the challenge vector $chall_2$ as input and reconstructs the full set of leaf nodes. Implementation details are provided in Section 4.2.1.

Part II

CROSS

In this part, we introduce classic zero-knowledge protocols in the form of sigma protocols, and demonstrate how to turn an interactive proof into a non-interactive proof using the Fiat-Shamir transform. We will show how the interactive CROSS-ID protocol works, as well as the implementation of CROSS-ID after Fiat-Shamir has been applied. A quick explanation of the security definitions for CROSS is given along with the choice of parameters. Before showing the CROSS implementation, the implementation of the tree algorithms will be shown and explained for the different variants: small, balanced, and fast.

3 PROTOCOL

3.1 ZERO-KNOWLEDGE PROTOCOLS

A classic example of a zero-knowledge protocol would be a sigma protocol. Sigma protocols are known as three-pass protocols. The Prover and the Verifier have some shared instance of a problem x. The prover also has knowledge of a secret, or is able to calculate it. The prover wishes to prove their knowledge of this secret without revealing it to the verifier directly [16]. Figure 3.1 shows the form of a sigma protocol, which also gives the protocol its characteristic name.

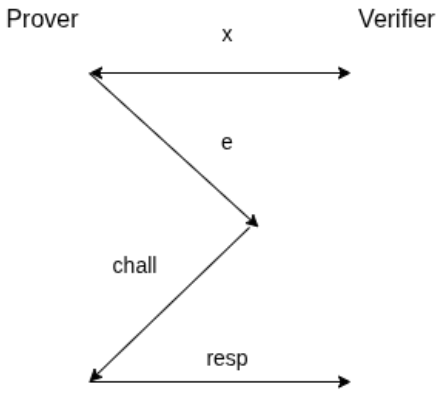

Figure 3.1: Setup of a sigma protocol

A sigma protocol must adhere to three properties: correctness, special soundness, and honest-verifier zero-knowledge [16].

Correctness specifies that a prover with the correct secret would always be able to make the verifier accept.
Special soundness states that, given any pair of accepting protocol executions with the same initial message from the prover, but with a different challenge from the verifier. It would be possible to efficiently compute the secret.
Finally, honest-verifier zero-knowledge specifies that the protocol can be simulated without the secret if we assume the challenge is picked at random.

This simulation is usually performed by picking the challenge and the response at random in the beginning, then it would be possible to calculate the initial message e such that the verifier accepts. This should give the same probability distribution as an actual execution

of the protocol, however, it is only possible because the simulator can choose the order in which the values are generated [16]. We denote ℓ as the number of possible challenge values. Since a challenge response on its own does not provide any secret information, a dishonest prover can pass the protocol with a probability of $\frac{1}{\ell}$. To avoid this being problematic, we can simply repeat the protocol for multiple rounds until the chance of a dishonest prover passing is negligible.

If a verifier obtains both challenge responses for a challenge, they would be able to extract the secret following the special soundness property. Therefore, it is important that the verifier never obtains both challenge responses if zero-knowledge is to be upheld. If the randomness for the first message from the prover is faulty or the protocol is run multiple times, causing e to be chosen multiple times, then a dishonest verifier could choose a different challenge value than previously and thereby learn part or all of the secret.

The value e is usually a binding to randomness or, in the case of our signature scheme, commitments to random values formed by hashing [16]. Essentially, a commitment must be both binding and hiding [17]. Various degrees of strength exist for these properties, such as computational and unconditional. Commitments based on hashing, if the hash function is assumed to be collision-resistant, are unconditionally hiding and computationally binding [17]. This is because the committer can change their commitment if they are given unbounded computing power as a collision could be found. The hash function used in CROSS is already assumed to follow the Random Oracle Model as described in Section 2.3. Since random oracles are collision-intractable, commitments in the scheme can therefore be seen as binding. To prevent an adversary from guessing the committed value by calculating the hash for all possible values, randomness is used. Randomness can be appended to the value before hashing to make the search space larger. The randomness can then be revealed along with the value if the commitment needs to be opened.

The underlying protocol for CROSS called CROSS-ID is not built directly as a sigma protocol, instead it is a 5-pass protocol involving an extra challenge and response step. Although these protocols differ, the inspiration from the classic sigma protocol setup is clear. One of the challenges is a binary challenge, while the other is in $\mathbb{F}_p^*$. Figure 3.2 shows the communication steps of CROSS-ID.

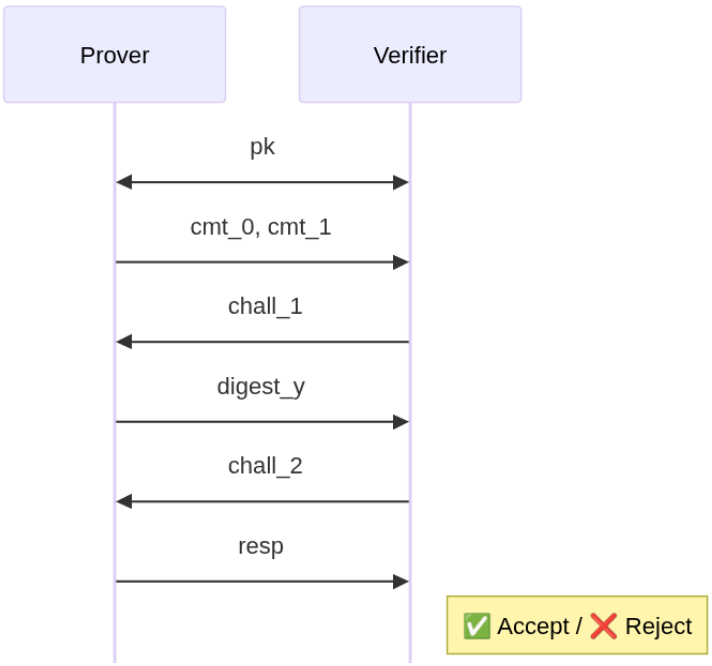

Figure 3.2: The interactions in the CROSS-ID protocol

3.2 CROSS-ID

The CROSS-ID is a q2-identification scheme, since $\mathsf{chall}_1 \in \mathbb{F}_p^*$ and $\mathsf{chall}_2 \in \{0, 1\}$, where the challenge spaces are $|\mathsf{chall}_1| = q = p - 1$ and $|\mathsf{chall}_2| = 2$, following the definition of a q2-identification scheme from Chen et al. [13]. The secret is the restricted vector $\mathbf{e} \in \mathbb{E}^n$, where $\mathbf{e} \in G$ such that the syndrome equation is satisfied $\mathbf{s} = \mathbf{e}\mathbf{H}^\top$. The verifier will either ask for a proof that the syndrome equation is satisfied when sending $\mathsf{chall}_2 = 0$ or ask for a proof that the secret vector is restricted by sending $\mathsf{chall}_2 = 1$. To give such a proof, the prover applies a linear transitive map $\mathbf{v} : \mathbb{E}^n \to \mathbb{E}^n$, and $\mathbf{v} : G \to G$. This can be done efficiently, since $\mathbb{E}$ is closed under multiplication [5]. The following protocol will be shown for R-SDP(G) , and this also includes R-SDP by setting $G = \mathbb{E}^n$.

To generate the commitments, the prover samples a random $\mathbf{e}' \in G$ and $\mathbf{u}' \in \mathbb{F}_p^n$ from a CSPRNG using a seed. The prover sets $\mathsf{cmt}_0 = \mathsf{Hash}((\mathbf{v} \star \mathbf{u}'\mathbf{H}^\top) \| \mathbf{v})$. For the second commitment, we hash the concatenation of $\mathbf{u}'$ and $\mathbf{e}'$, which gives $\mathsf{cmt}_1 = \mathsf{Hash}(\mathbf{u}' \| \mathbf{e}')$.
The next step is for the verifier to generate a challenge and for the prover to give a response. The challenge is sampled by the verifier as $\mathsf{chall}_1 \in \mathbb{F}_p^*$, which is sent to the prover. The prover will then compute the response as $\mathbf{y} = \mathbf{u}' + \mathsf{chall}_1\mathbf{e}' = \mathbf{v}^{-1} \star (\mathbf{u} + \mathsf{chall}_1\mathbf{e}) = \mathbf{v}^{-1} \star \mathbf{u} + \mathsf{chall}_1\mathbf{v}^{-1} \star \mathbf{e}$. The prover could send $\mathbf{y}$ directly to the verifier, but to reduce the communication cost the prover sends $\mathsf{Hash}(\mathbf{y})$.

For the second challenge and response, the verifier will sample a challenge $\mathsf{chall}_2 \xleftarrow{\$} \{0,1\}$. The response generated by the prover's will either verify cmt_0 by sending $(\mathbf{y}, \mathbf{v})$ or it will verify cmt_1 by sending seed, which was used to compute $\mathbf{e}', \mathbf{u}'$.

For the verification step, the verifier needs to verify cmt_0 from $\mathbf{H}, \mathbf{s}$ and the response from the prover $\mathbf{y}, \mathbf{v}$. The first step for the verifier is to check that $\mathbf{v} \in \mathsf{G}$ and that $\mathsf{digest}_\mathbf{y} = \mathsf{Hash}(\mathbf{y})$, if this passes, then the prover can recover cmt_0 by computing $\mathsf{cmt}_0 = \mathsf{Hash}(\mathbf{v} \star \mathbf{y}\mathbf{H}^\top - \mathsf{chall}_1\mathbf{s} \| \mathbf{v})$, this is possible since $\mathbf{v} \star \mathbf{y} = \mathbf{u} + \mathsf{chall}_1\mathbf{e}$.
If the verifier instead receives the seed to generate $\mathbf{e}', \mathbf{u}'$, then the verifier checks that $\mathsf{digest}_\mathbf{y} = \mathsf{Hash}(\mathbf{u}' + \mathsf{chall}_1\mathbf{e}')$, if this passes, then cmt_1 can be recovered by computing $\mathsf{cmt}_1 = \mathsf{Hash}(\mathbf{u}' \| \mathbf{e}')$. The CROSS-ID protocol is shown in Figure 3.3.

Secret Key: $\mathbf{e} \in G$
Public Key: $G \subseteq \mathbb{E}^n$, $\mathbf{H} \in \mathbb{F}_p^{(n-k)\times n}$, $\mathbf{s} = \mathbf{e}\mathbf{H}^\top \in \mathbb{F}_p^{n-k}$

PROVER		**VERIFIER**
// Sampling Seed to compute $\mathbf{e}', \mathbf{u}'$		
$\mathsf{Seed} \xleftarrow{\$} \{0,1\}^\lambda$		
$(\mathbf{e}', \mathbf{u}') \leftarrow \mathsf{CSPRNG}(\mathsf{Seed})$		
// with co-domain $G \times \mathbb{F}_p^n$		
$\mathbf{v} \leftarrow \mathbf{e} \star (\mathbf{e}')^{-1}$		
$\mathbf{u} \leftarrow \mathbf{v} \star \mathbf{u}'$		
$\mathbf{s}' \leftarrow \mathbf{u}\mathbf{H}^\top$		
// Computing commitments		
$\mathsf{cmt}_0 \leftarrow \mathsf{Hash}(\mathbf{s}' \Vert \mathbf{v})$		
$\mathsf{cmt}_1 \leftarrow \mathsf{Hash}(\mathbf{u}' \Vert \mathbf{e}')$	$\xrightarrow{\mathsf{cmt}_0, \mathsf{cmt}_1}$	
		// Sampling first challenge
	$\xleftarrow{\mathsf{chall}_1}$	$\mathsf{chall}_1 \xleftarrow{\$} \mathbb{F}_p^*$
// Computing first response		
$\mathbf{y} \leftarrow \mathbf{u}' + \mathsf{chall}_1 \mathbf{e}'$		
$\mathsf{digest}_y \leftarrow \mathsf{Hash}(\mathbf{y})$	$\xrightarrow{\mathsf{digest}_y}$	
		// Sampling second challenge
	$\xleftarrow{\mathsf{chall}_2}$	$\mathsf{chall}_2 \xleftarrow{\$} \{0,1\}$
// Computing second response		
If $\mathsf{chall}_2 = 0$, $\mathsf{resp} \leftarrow (\mathbf{y}, \mathbf{v})$		
If $\mathsf{chall}_2 = 1$, $\mathsf{resp} \leftarrow \mathsf{Seed}$	$\xrightarrow{\mathsf{resp}}$	
		// Verification
		If $\mathsf{chall}_2 = 0$:
		$\mathbf{y}' \leftarrow \mathbf{v} \star \mathbf{y}$
		$\mathbf{s}' \leftarrow \mathbf{y}'\mathbf{H}^\top - \mathsf{chall}_1 \mathbf{s}$
		Accept if:
		1) $\mathsf{Hash}(\mathbf{y}) = \mathsf{digest}_y$
		2) $\mathsf{Hash}(\mathbf{s}' \Vert \mathbf{v}) = \mathsf{cmt}_0$
		3) $\mathbf{v} \in G$
		If $\mathsf{chall}_2 = 1$:
		$(\mathbf{e}', \mathbf{u}') \leftarrow \mathsf{CSPRNG}(\mathsf{Seed})$
		// with co-domain $G \times \mathbb{F}_p^n$
		$\mathbf{y} \leftarrow \mathbf{u}' + \mathsf{chall}_1 \mathbf{e}'$
		Accept if:
		1) $\mathsf{Hash}(\mathbf{y}) = \mathsf{digest}_y$
		2) $\mathsf{Hash}(\mathbf{u}' \Vert \mathbf{e}') = \mathsf{cmt}_1$

Figure 3.3: CROSS-ID

3.3 FIAT–SHAMIR TRANSFORM

The Fiat-Shamir transform is a common heuristic, used to transform public coin interactive proofs into signature schemes. A public coin interactive proof is a protocol in which the random choices made by the verifier are made public [41]. In CROSS, this would be the two challenge vectors $\mathsf{chall}_1, \mathsf{chall}_2$. The main idea for transforming an identification protocol into a signature scheme is to replace the challenge-communication steps with a hash function. Using the Fiat-Shamir transform, a Sigma protocol can be transformed into a digital signature scheme that is EUF-CMA secure in the Random Oracle Model (ROM) [6]. When proving the security of a signature scheme based on a multi-round interactive proof, the hash call H_i in each round is considered as different Random Oracles. In practice, this can be achieved by using a single hash function with different domain separator constants for each round, as specified in Section 2.3.
To see how the Fiat-Shamir transform works, we will transform a simple public coin interactive proof into a signature scheme. For this setup, we have a prover and a verifier (P, V). The prover initiates the protocol by sampling a problem instance $x \in \{0,1\}^n$. The interactive proof goes as follows [41]:

- P sends the first message $\alpha \leftarrow P(x)$, $\alpha \in \{0,1\}^a$
- V sends a response $\beta \leftarrow \{0,1\}^b$
- P sends the final message $\gamma \leftarrow P(x, \alpha, \beta)$, $\gamma \in \{0,1\}^c$
- V decides to accept or reject according to an algorithm $V(x, \alpha, \beta, \gamma)$

The goal of the transformation is to allow the prover P to send a single message π and to let the verifier accept or reject based only on this message. This is done by replacing the verifier's uniformly random message β with something the prover can compute on their own. We cannot let the prover sample β themselves as the prover could be malicious and choose β such that the signature is accepted despite not knowing the secret. Instead, the prover will compute β as a function of the transcript of the protocol so far, this is done using a hash function [41]. This removes the prover's ability to choose a specific challenge and cheat that way. The result of this transformation will look like this:

- P computes $\alpha \leftarrow P(x)$
- P computes $\beta \leftarrow H(x, \alpha)$
- P computes $\gamma \leftarrow P(x, \alpha, \beta)$
- P sends (α, γ) to V
- V computes $\beta \leftarrow H(x, a)$ and accepts iff $V(x, \alpha, \beta, \gamma)$ accepts

3.4 CROSS PARAMETERS

CROSS parameters are chosen to meet NIST's security levels 1,3, and 5. These levels are defined as the cost of breaking AES with 128, 192, or 256 bit keys [5]. The code parameters p, n, k and the restriction parameters z, m are chosen accordingly [5]. The parameters t, w are chosen to determine the zero-knowledge protocol's security level optimally while avoiding possible forgery attacks. It is preferred to have $\mathsf{chall}_2 = 1$ since fewer bits are transmitted, because $|\mathsf{seed}| \leqslant |(\mathbf{y}, \mathbf{v})| = \lambda \leqslant n\lceil\log_2(p)\rceil + n\lceil\log_2(z)\rceil$ [5]. For the R-SDP(G) case, a similar optimization is used. The only difference is that we are sending $(\mathbf{y}, \overline{\mathbf{v}}_G)$ instead, so the calculation becomes: $|\mathsf{seed}| \leqslant |(\mathbf{y}, \overline{\mathbf{v}}_G)| = \lambda \leqslant n\lceil\log_2(p)\rceil + m\lceil\log_2(z)\rceil$ [5].
In the implementation, packing is utilized to further minimize the size of these bit strings, upon which the calculations are based. However, despite the larger size, it is necessary to have some amount of 0's in chall_2 so an attacker cannot easily make the protocol pass by always preparing the $\mathsf{chall}_2 = 1$ response, as we know from the honest-verifier zero-knowledge property that a protocol can be simulated for one response without knowing the secret. If this was not the case, some new knowledge would otherwise have been obtained.

Algorithm and Security Category	**Variant**	p	z	n	k	m	t	w	Pri. Key Size (B)	Pub. Key Size (B)	Signature Size (B)
CROSS-R-SDP 1	fast	127	7	127	76	-	157	82	32	77	18432
	balanced	127	7	127	76	-	256	215	32	77	13152
	small	127	7	127	76	-	520	488	32	77	12432
CROSS-R-SDP 3	fast	127	7	187	111	-	239	125	48	115	41406
	balanced	127	7	187	111	-	384	321	48	115	29853
	small	127	7	187	111	-	580	527	48	115	28391
CROSS-R-SDP 5	fast	127	7	251	150	-	321	167	64	153	74590
	balanced	127	7	251	150	-	512	427	64	153	53527
	small	127	7	251	150	-	832	762	64	153	50818
CROSS-R-SDP(G) 1	fast	509	127	55	36	25	147	76	32	54	11980
	balanced	509	127	55	36	25	256	220	32	54	9120
	small	509	127	55	36	25	512	484	32	54	8960
CROSS-R-SDP(G) 3	fast	509	127	79	48	40	224	119	48	83	26772
	balanced	509	127	79	48	40	268	196	48	83	22464
	small	509	127	79	48	40	512	463	48	83	20452
CROSS-R-SDP(G) 5	fast	509	127	106	69	48	300	153	64	106	48102
	balanced	509	127	106	69	48	356	258	64	106	40100
	small	509	127	106	69	48	642	575	64	106	36454

Table 3.1: Overview of parameters for R-SDP and R-SDP(*G*) across all variants and security levels

3.5 SECURITY OF CROSS

3.5.1 EUF-CMA

Existential Unforgeability under Chosen-Message Attack (EUF-CMA) is a security definition for signature schemes. NIST generally requires at least EUF-CMA security for signature schemes in their Post-Quantum calls [1]. For the setup, an adversary has access to the public key and a signing oracle. The adversary can request as many messages to be signed as they want, by repeatedly making calls to the oracle. The adversary must output a message and signature that verifies under the public key, and the message cannot be one of the messages previously submitted to the oracle [23].

Strong Existential Unforgeability under Chosen Message Attack (SUF-CMA) is a stronger definition than EUF-CMA. In SUF-CMA, the adversary can succeed with a message previously signed by the oracle as long as the signature for the message is not the same as the one received from the oracle. Essentially, the message and signature pair must be unique from previous interactions. If a signature is still valid when small changes have been applied to it, then the scheme would not be SUF-CMA [23].

CROSS utilizes a proof such that, if the interactive proof has both special soundness and honest-verifier zero-knowledge then applying the Fiat-Shamir transform creates an EUF-CMA secure non-interactive protocol [6]. CROSS has not been proven to follow SUF-CMA.

3.5.2 Padding Attack

It is important to note that due to the optimizations used in the tree structure, a check of the padding of the `proof` and `path`, should be implemented as seen in version 2 of the CROSS codebase [12]. This is not directly noted in the specification's pseudo-code. The padding consists entirely of zero bytes, and the check must verify that all bytes after the last `published` seed are, in fact, zero bytes. This is done by initializing a zero byte, and then performing OR operations with all the remaining bytes, if a single positive bit is present, then the `Verify` algorithm will reject the signature.

Although producing a signature on a message already requested from the oracle does not directly break EUF-CMA, which is the security claimed by CROSS, then it breaks the stronger SUF-CMA, since a unique pair of message and a new signature is made, even though a signature for the message would have to be requested from the oracle first [23]. Implementing the padding on `path` and `proof` is quite trivial and is described in Section 4.2.1. The mechanism for checking that the padding is valid does not increase the worst-case bounds.

4 IMPLEMENTATION

Our implementation of CROSS is available on GitHub at https://github.com/Rasmus3650/CROSS-Go. It is based on version 2.1 of the CROSS specification [5]. The implementation follows an object-oriented style by grouping all necessary parameters such as n, t, w, and TreeParams, which is itself a structured set of parameters. Functions related to the scheme such as `KeyGen, Sign, Verify, SeedLeaves` and `RecomputeRoot` are defined as methods on the structure. Although Go does not support traditional object-oriented features like classes or inheritance, this approach achieves similar goals: better code organization, a significant reduction in the number of parameters passed between functions, and improved safety by keeping parameters tied to a specific instance of the scheme. An example of how the implementation should be used can be seen in Listing 4.1, where the CROSS instance is initialized as balanced R-SDP with security level 1.

Listing 4.1: Example Usage of the CROSS implementation

```
func main() {
        // Initialize the CROSS instance
        cross, err := vanilla.NewCROSS(common.RSDP_1_BALANCED)
        if err != nil {
                panic(err)
        }
        // Generate keys
        keys, err := cross.KeyGen()
        if err != nil {
                panic(err)
        }
        // Sign a message
        msg := []byte("Hello, world!")
        sig, err := cross.Sign(keys.Sk, msg)
        if err != nil {
                panic(err)
        }
        // Verify the signature
        ok, err := cross.Verify(keys.Pk, msg, sig)
        if err != nil {
                panic(err)
        }
        if !ok {
                panic("Signature verification failed")
        }
}
```

4.1 CSPRNG – SHAKE

The CROSS scheme relies on a CSPRNG, which can be called in different contexts, like having output in $\mathbb{F}_p$, $\mathbb{F}_z$ or in a weighted Boolean vector. The simplest of the CSPRNG calls is the one where we simply need it to output a byte vector $\text{out} \in \{0, 255\}^n$. The standard CSPRNG call takes a seed, an output length, and a domain separator constant. In the Go implementation, there are two versions of the standard CSPRNG function, the function `CSPRNG` outputs the byte array, and `CSPRNG_prime` outputs the byte array and the state of the CSPRNG after the byte array has been sampled.

The standard `CSPRNG` is shown in Algorithm 4.2, the `CSPRNG_prime` is exactly the same, but the `shake` object is returned along with the array. Every time the CSPRNG is called using a state, it will take in the state and output an updated state, along with the sampled values. This is necessary since the reference implementation utilizes and updates a shared state across multiple CSPRNG calls. Without this approach, it would not be possible to generate compliant signatures. There is an if-statement in the `CSPRNG` function that determines whether to use `Shake128` or `Shake256` depending on the security level of the CROSS instance.

Listing 4.2: Standard CSPRNG

```
func (c *CROSS[T, P]) CSPRNG(seed []byte, output_len int, dsc uint16) []
    byte {
        var shake sha3.ShakeHash
        if c.ProtocolData.Level() == 1 {
                shake = sha3.NewShake128()
        } else {
                shake = sha3.NewShake256()
        }
        shake.Write(seed)
        // Prepare the dsc value in little-endian byte order
        dscOrdered := []byte{
                byte(dsc & 0xff),        // low byte
                byte((dsc >> 8) & 0xff), // high byte
        }

        shake.Write(dscOrdered)
        output := make([]byte, output_len)
        shake.Read(output)
        return output
}
```

There are some places where CROSS requires the output to be in a finite field like $\mathbb{F}_p$ or $\mathbb{F}_z$, to accomplish this, a type of rejection sampling is used. An example of a function that does this is `CSPRNG_fp_vec` or `CSPRNG_fp_mat`, the only real difference between these are the output

lengths. There is also a variant of `CSPRNG_fp_mat` called `CSPRNG_fp_mat_prime`, which does the same thing as `CSPRNG_fp_mat` but instead of taking a seed as input, it takes the state of the CSPRNG which is encapsulated in the `shake` object. The function `CSPRNG_fp_vec` is shown in Algorithm 4.3. A sub-buffer with the immediate CSPRNG output is utilized to perform the various bit operations. The important part of rejecting a sample outside of $\mathbb{F}_p$ is shown in lines 28-30.

Listing 4.3: CSPRNG function that outputs elements in $\mathbb{F}_p$

```
1  func (c *CROSS[T, P]) CSPRNG_fp_vec(seed []byte) []byte {
2          res := make([]byte, c.ProtocolData.N)
3          FP_ELEM_mask := (uint8(1) << BitsToRepresent(uint(c.ProtocolData.
               P-1))) - 1
4          dsc := uint16(2*c.ProtocolData.T - 1)
5          BITS_FOR_P := BitsToRepresent(uint(c.ProtocolData.P - 1))
6          CSPRNG_buffer := c.CSPRNG(seed, int(RoundUp(uint(c.ProtocolData.
               BITS_N_FP_CT_RNG), 8)/8), dsc)
7          placed := 0
8          sub_buffer := uint64(0)
9          for i := 0; i < 8; i++ {
10                 sub_buffer |= uint64(CSPRNG_buffer[i]) << uint64(8*i)
11         }
12         bits_in_sub_buf := 64
13         pos_in_buf := 8
14         pos_remaining := len(CSPRNG_buffer) - pos_in_buf
15         for placed < c.ProtocolData.N {
16                 if bits_in_sub_buf <= 32 && pos_remaining > 0 {
17                         refresh_amount := int(math.Min(4, float64(
                               pos_remaining)))
18                         refresh_buf := uint32(0)
19                         for i := 0; i < refresh_amount; i++ {
20                                 refresh_buf |= uint32(CSPRNG_buffer[
                                       pos_in_buf+i]) << byte(8*i)
21                         }
22                         pos_in_buf += refresh_amount
23                         sub_buffer |= uint64(refresh_buf) << uint64(
                               bits_in_sub_buf)
24                         bits_in_sub_buf += 8 * refresh_amount
25                         pos_remaining -= refresh_amount
26                 }
27                 res[placed] = uint8(uint8(sub_buffer) & FP_ELEM_mask)
28                 if res[placed] < uint8(c.ProtocolData.P) {
29                         placed++
30                 }
31                 sub_buffer >>= BITS_FOR_P
32                 bits_in_sub_buf -= BITS_FOR_P
33         }
34         return res
35 }
```

Finally, for the fixed-weight challenge vector, we want to output a weighted Boolean vector, with w positive entries, this is done by the `Expand_digest_to_fixed_weight` function. In this function a Fisher-Yates shuffle is utilized [5]. Fisher-Yates shuffling essentially takes a vector with predefined elements and selects random positions using a CSPRNG. The random numbers are selected within the range of $t - curr$ which represents the remaining unsorted part of the vector [5]. These random numbers become a chosen position, we then swap the Boolean value at the chosen position to the current position, and then add 1 to the current position. The implementation for `Expand_digest_to_fixed_weight` is shown in Listing 4.4.

Listing 4.4: CSPRNG call that outputs a weighted Boolean vector

```
func (c *CROSS[T, P]) Expand_digest_to_fixed_weight(digest []byte) []bool {
	fixed_weight_string := make([]bool, c.ProtocolData.T)
	dsc_csprng_b := uint16(3*c.ProtocolData.T)
	CSPRNG_buffer := c.CSPRNG(digest, int(RoundUp(uint(c.ProtocolData.BITS_CWSTR_RNG), 8)/8), dsc_csprng_b)
	for i := 0; i < c.ProtocolData.W; i++ {
		fixed_weight_string[i] = true
	}
	sub_buffer := uint64(0)
	for i := 0; i < 8; i++ {
		sub_buffer |= uint64(CSPRNG_buffer[i]) << uint64(8*i)
	}
	bits_in_sub_buf := 64
	pos_in_buf := 8
	pos_remaining := len(CSPRNG_buffer) - pos_in_buf
	curr := 0
	for curr < c.ProtocolData.T {
		if bits_in_sub_buf <= 32 && pos_remaining > 0 {
			refresh_amount := int(math.Min(4, float64(pos_remaining)))
			refresh_buf := uint32(0)
			for i := 0; i < refresh_amount; i++ {
				refresh_buf |= uint32(CSPRNG_buffer[pos_in_buf+i]) << byte(8*i)
			}
			pos_in_buf += refresh_amount
			sub_buffer |= uint64(refresh_buf) << uint64(bits_in_sub_buf)
			bits_in_sub_buf += 8 * refresh_amount
			pos_remaining -= refresh_amount
		}
		bits_for_pos := BitsToRepresent(uint(c.ProtocolData.T - 1 - curr))
		pos_mask := (uint64(1) << uint64(bits_for_pos)) - 1
		candidate_pos := uint16(sub_buffer & pos_mask)
		if candidate_pos < uint16(c.ProtocolData.T-curr) {
			dest := curr + int(candidate_pos)
			tmp := fixed_weight_string[curr]
			fixed_weight_string[curr] = fixed_weight_string[dest]
			fixed_weight_string[dest] = tmp
			curr++
		}
		sub_buffer >>= uint64(bits_for_pos)
		bits_in_sub_buf -= bits_for_pos
	}
	return fixed_weight_string
}
```

4.2 TREE ALGORITHMS

The tree algorithms were briefly introduced in Section 2.4. In this section, we provide a detailed account of the implementation of the seed and Merkle trees used in the CROSS scheme. Before presenting the specific algorithms, we define a structure to represent the shape and properties of the unbalanced trees. This is done using a helper structure called `TreeParams`, which is nested within the main `CROSS` structure. The `TreeParams` structure contain six fields that describe the tree, including its depth, branching, and indexing logic. The layout of this structure is shown in Table 4.1.

Field	Type	Description
`NPL`	`[]int`	Nodes per level
`LPL`	`[]int`	Leaves per level
`Off`	`[]int`	Offsets for parent/child computation
`LSI`	`[]int`	Leaves start indices
`NCL`	`[]int`	Number of consecutive leaves
`Total_nodes`	`int`	Total number of nodes in tree

Table 4.1: Structure of the `TreeParams` struct

4.2.1 Seed trees

As described in Section 2.4, there are two types of seed trees used in CROSS: small/balanced trees and fast trees. The Go implementation differentiates between these cases using a helper function that runs on the currently initialized `CROSS` instance. To construct the small and balanced trees, a two-dimensional byte array `[][]byte` is initialized to store the seeds of all nodes in the tree. The length of this array is determined by the `Total_nodes` field in the `TreeParams` structure. After initialization, the first entry in the array is set to the root seed, which is provided as a parameter. The tree is then expanded layer by layer.

For each non-leaf node, we compute its left and right child seeds. To ensure correctness, the loop skips over the leaf nodes, as they should not have children. For a given node, the child indices are computed using tree-specific indexing logic [14], adjusting the ordinary way of calculating children and parents with our offsets. The seed of the current node is concatenated with the public salt, that is, the input to the CSPRNG is $\mathcal{T}[\mathsf{node}]||\mathsf{salt}$, where $\mathcal{T}$ is the byte array that contains all the seeds of the nodes, and $\mathcal{T}[\mathsf{node}]$ refers to the seed of the current node. This input is passed to the CSPRNG, which outputs a 2λ-bit string. A domain separation constant of node is used as part

of the CSPRNG input to ensure that each node is sampled from its own domain, as seen in Section 2.3. The resulting bit string is split into two λ-bit seeds, which are then assigned to the left and right child nodes, respectively. Using a helper function `Leaves`, the leaf nodes can be extracted from the tree, and this output will be returned by the `SeedLeaves` function. The pseudo-code for `SeedLeaves` is shown in Algorithm 1, and the actual implementation of `Leaves` is shown in Listing 4.5.

Algorithm 1 SeedLeaves(`seed`, `salt`)

Inputs:
`seed`: Initial root seed
`salt`
Auxiliary Data:
`TreeParams`: Tree parameters (`TotalNodes`, `LPL`, `NPL`, etc.)
λ: Security parameter (in bits)
`CSPRNG`: Cryptographically secure PRNG
`LeftChild(node, level)`: Left child index function
Output:
$\mathcal{T}$: Array of derived node seeds

1: $\mathcal{T} \leftarrow$ new array of byte arrays of length `TreeParams.TotalNodes`
2: $\mathcal{T}[0] \leftarrow$ `seed`
3: $start\_node \leftarrow 0$
4: **for** $level \leftarrow 0$ **to** $\lceil \log_2(t) \rceil - 1$ **do**
5: **for** $i \leftarrow 0$ **to** `NPL`$[level] -$ `LPL`$[level] - 1$ **do**
6: $node \leftarrow start\_node + i$
7: $left \leftarrow$ `LeftChild`$(node, level)$
8: $right \leftarrow left + 1$
9: $hash \leftarrow$ `CSPRNG` $-\{0,1\}^{\lambda} \times \{0,1\}^{\lambda}(\mathcal{T}[node] \| \texttt{salt}, node)$
10: $\mathcal{T}[left] \leftarrow hash[:\lambda]$
11: $\mathcal{T}[right] \leftarrow hash[\lambda:]$
12: **end for**
13: $start\_node \leftarrow start\_node +$ `NPL`$[level]$
14: **end for**
15: **return** `leaves`$(\mathcal{T})$

Listing 4.5: Leaves function used to traverse the tree and extract leaf nodes

```
func (c *CROSS[T, P]) Leaves(tree [][]byte) [][]byte {
        result := [][]byte{}
        for i := 0; i < len(c.TreeParams.LSI); i++ {
                index := c.TreeParams.LSI[i]
                for j := 0; j < c.TreeParams.NCL[i]; j++ {
                        result = append(result, tree[index+j])
                }
        }
        return result
}
```

To construct the seed tree for the fast variant of CROSS, we begin similarly to the small/balanced tree case by initializing a `[][]byte` array to hold the seeds of all nodes. The root seed is placed at index 0. In the fast version, the root node always expands to exactly four children. To generate these seeds, the concatenation $\mathcal{T}[0]||\mathsf{salt}$ is given as input to the CSPRNG along with a domain separation constant of 0. The CSPRNG outputs a 4λ-bit string, which is split into four λ-bit seeds. These are assigned to the four child nodes of the root. Next, an integer array of length 4 is initialized to determine how many children each of the second-layer nodes should have. This depends on the number of leaves t, and falls into one of four cases:

- If t is divisible by 4, each node receives exactly $\frac{t}{4}$ children.
- If $t \bmod 4 = 1$, the first node receives $\frac{t}{4} + 1$ children, and the remaining three nodes receive $\frac{t}{4}$ each
- If $t \bmod 4 = 2$, the first two nodes each receive $\frac{t}{4} + 1$ children, and the last two receive $\frac{t}{4}$
- If $t \bmod 4 = 3$, the first three nodes each receive $\frac{t}{4} + 1$ children, and the last node receives $\frac{t}{4}$.

This ensures that the t leaf nodes are evenly distributed across the second-layer of parent nodes, with the remaining children added from left to right [5]. After determining the number of children each second-layer node should have, the second-layer nodes are expanded. For each of the four nodes, their seed is concatenated with the salt and passed to the CSPRNG, using a domain separation constant $\mathsf{dsc} \in \{1, 2, 3, 4\}$, which increments with each iteration. The CSPRNG outputs $\mathsf{children}[i] \cdot \lambda$ bits, which are split into $\mathsf{children}[i]$ separate λ-bit seeds. These newly generated seeds are stored in the leaf nodes. The pseudo-code for `SeedLeaves` for the fast variant is provided in Algorithm 2.

Algorithm 2 SEEDLEAVES(`seed`, `salt`)

Inputs:
`seed`: Initial root seed
`salt`
Auxiliary Data:
`TreeParams.TotalNodes`
λ: Security parameter (in bits)
t: Number of leaves
`CSPRNG`: Cryptographically secure PRNG
Output:
$\mathcal{T}$: Array of derived leaf seeds

$\mathcal{T} \leftarrow$ new array of byte arrays of length `TreeParams.TotalNodes`
$\mathcal{T}[0] \leftarrow$ `seed`
// Derive four intermediate seeds from root
$quadSeeds \leftarrow$ `CSPRNG`$(\mathcal{T}[0] \parallel$ `salt`$) \in \{0,1\}^{4\lambda}$
for $i \leftarrow 0$ **to** 3 **do**
    $\mathcal{T}[i+1] \leftarrow quadSeeds[i\lambda : (i+1)\lambda]$
end for
// Compute number of children each intermediate node has
$children \leftarrow [0,0,0,0]$
if $t \bmod 4 = 0$ **then**
    $children[i] \leftarrow T/4$ **for all** i
else if $t \bmod 4 = 1$ **then**
    $children[0] \leftarrow \lfloor T/4 \rfloor + 1$
    $children[1..3] \leftarrow \lfloor T/4 \rfloor$
else if $t \bmod 4 = 2$ **then**
    $children[0], children[1] \leftarrow \lfloor T/4 \rfloor + 1$
    $children[2], children[3] \leftarrow \lfloor T/4 \rfloor$
else if $t \bmod 4 = 3$ **then**
    $children[0], children[1], children[2] \leftarrow \lfloor T/4 \rfloor + 1$
    $children[3] \leftarrow \lfloor T/4 \rfloor$
end if
// Derive leaf seeds from intermediate seeds
$result \leftarrow \emptyset$
$ctr \leftarrow 0$
for $i \leftarrow 0$ **to** 3 **do**
    $ctr \leftarrow ctr + 1$
    $hash \leftarrow$ `CSPRNG`$(\mathcal{T}[i+1] \parallel$ `salt`$, ctr) \in \{0,1\}^{children[i] \cdot \lambda}$
    for $j \leftarrow 0$ **to** $children[i] - 1$ **do**
        append $hash[j\lambda : (j+1)\lambda]$ to $result$
    end for
end for
return $result$

The second required operation on seed trees is the `SeedPath` function. This function constructs the full seed tree by internally calling `BuildTree`, which is identical to `SeedLeaves`, except that it returns the entire tree instead of only the leaves. The set of nodes that needs to be revealed is determined by the chall_2 vector, which is passed to the `computeNodesToPublish` function. For the balanced/small trees, the algorithm starts by traversing all leaf nodes, similar to how `Leaves` selects leaves, but filters the output by only selecting nodes for which the corresponding entry in the chall_2 Boolean vector is set to `true`.

Once all required leaf nodes have been selected, a compression step is applied to minimize the size of the signature. If both children of a given parent node are present in the output set, then they are removed and replaced by their parent. This process is repeated throughout the tree, allowing internal nodes to replace their children whenever possible. The `computeNodesToPublish` function outputs a Boolean vector that marks exactly which nodes should be included in the seed path. The `SeedPath` function then uses this vector to traverse the entire tree and collect only the necessary nodes to include in the signature. It is very important that the output seed path is stored in a fixed-size array initialized with zero bytes (padding). This ensures that the array is completely filled and that any unused space only contains zero bytes. Padding is essential to prevent trivial attacks on the signature [5]. This is possible because `RebuildLeaves` only processes the first $\mathsf{published}$ entries, where $\mathsf{published}$ is the number of published nodes after compression. This attack is described in more detail in Section 3.5.2.

The fast variant of `SeedPath` is simpler, it only requires traversing the leaf layer. The leaf nodes corresponding to `true` values in chall_2 are directly included. Since there is no compression in this case, no padding is necessary. The implementation of the `SeedPath` function is shown in Algorithm 3.

Algorithm 3 SEEDPATH(`seed`, `salt`, $\texttt{chall}_2$)

Inputs:
`seed`: Initial root seed
`salt`
$\texttt{chall}_2$: Boolean challenge vector
Auxiliary Data:
t: Number of leaves
`Variant`: Variable that stores the variant of the current instance
`Total_Nodes`: Total number of nodes in the path
Output:
Path: Path in the seed tree

1: **if** `Variant` == `Balanced` **or** `Variant` == `Small` **then**
2: $\mathcal{T} \leftarrow$ BuildTree(seed, salt)
3: $\mathcal{T}' \leftarrow$ ComputeNodesToPublish($\texttt{chall}_2$)
4: Path $\leftarrow \emptyset$
5: published $\leftarrow 0$
6: **for** i $\leftarrow 0$ **to** Total_Nodes **do**
7: **if** $\mathcal{T}'$[i] == True **then**
8: Path[published] $\leftarrow \mathcal{T}$[i]
9: published $\leftarrow$ published $+ 1$
10: **end if**
11: **end for**
12: **return** Path
13: **end if**
14: **if** `Variant` == `Fast` **then**
15: $\mathcal{T}' \leftarrow$ SeedLeaves(seed, salt)
16: Path $\leftarrow \emptyset$
17: **for** i $\leftarrow 0$ **to** t **do**
18: **if** $\texttt{chall}_2$[i] == True **then**
19: append $\mathcal{T}'$[i] to Path
20: **end if**
21: **end for**
22: **return** Path
23: **end if**

The final required operation on the seed tree is the reconstruction of the leaf nodes from a given `path`, `salt`, and challenge vector $\texttt{chall}_2$. The behavior of `RebuildLeaves` is different depending on whether the instance is of type small/balanced or fast. In the small and balanced case, the function first computes the Boolean node tree $\mathcal{T}'$ using `computeNodesToPublish`, based on the provided $\texttt{chall}_2$ vector. This tree indicates which nodes are part of the path and need to be expanded. An empty tree $\mathcal{T}$ of size `Total_nodes` is initialized to hold the seeds during reconstruction.

The tree is traversed level by level starting from level 1, since level 0 contains the root, which is not part of any path unless all leaf nodes are included, which never occurs. For each node at a given level, we check:

- If the node is marked in $\mathcal{T}'$ and its parent is not, then the node must have been part of the published path. Its corresponding seed is copied from the `path` array into $\mathcal{T}$, and the index of published nodes is incremented.
- If the node is marked and it is a non-leaf node, its left and right child seeds are deterministically expanded using the CSPRNG. The input to the CSPRNG is $\mathcal{T}[\mathsf{node}]||\mathsf{salt}$, and a domain separation constant of `node` is used. The 2λ-bit output is split into two λ-bit child seeds, which are placed in their respective positions in $\mathcal{T}$, These child nodes are then marked in $\mathcal{T}'$ so their children can be expanded in the next iteration, if needed.

After traversing all layers, the leaf nodes are extracted from $\mathcal{T}$ using the `Leaves` function. Before returning, a final check is performed on the remaining entries in the `path` array, the entries beyond the number of nodes published. Each byte in these unused entries is checked to ensure it is zero. If any non-zero byte is found, the function returns `false` to indicate signature tampering. This zero padding is crucial to protect against trivial padding attacks, where an attacker could append junk data to the path while still having a valid signature.

In the fast variant of this function, the reconstruction is simpler. The function allocates an array of size t, one entry for each leaf. It then iterates over the leaf indices, and if the corresponding chall_2 entry is `true`, the node is reconstructed from the next available seed in the `path`. Otherwise, an empty λ-bit array is assigned to that position. Since no compression is used in the fast case, there is no need for tree traversal or verification of the padding. The implementation of the `RebuildLeaves` function is shown in Algorithm 4.

Algorithm 4 REBUILDLEAVES(Path, salt, chall_2)

Inputs:
Path: Initial root seed
salt
chall_2: Boolean challenge vector
Auxiliary Data:
t: Number of leaves
Variant: Variable that stores the variant of the current instance
λ: Security parameter (in bits)
CSPRNG: Cryptographically secure PRNG
Total_Nodes: Total number of nodes in the path
Output:
result: Leaves of the seed tree
ok: Boolean value to check if the padding has been modified

1: **if** Variant == Balanced **or** Variant == Small **then**
2: $\mathcal{T}' \leftarrow$ ComputeNodesToPublish(chall_2)
3: $\mathcal{T} \leftarrow \emptyset$
4: start_node $\leftarrow 1$
5: pub_nodes $\leftarrow 0$
6: **for** level $\leftarrow 1$ **to** $\lceil \log_2(t) \rceil$ **do**
7: **for** i $\leftarrow 0$ **to** NPL[level] $- 1$ **do**
8: node $\leftarrow$ start_node $+$ i
9: parent $\leftarrow$ Parent(node, level)
10: left_child $\leftarrow$ LeftChild(node, level)
11: right_child $\leftarrow$ left_child $+ 1$
12: **if** $\mathcal{T}'$[node] $==$ True **and** $\mathcal{T}'$[parent] $==$ False **then**
13: $\mathcal{T}$[node] $\leftarrow$ Path[pub_nodes]
14: pub_nodes $\leftarrow$ pub_nodes $+ 1$
15: **end if**
16: **if** indexes[node] **and** i $<$ NPL[level] $-$ LPL[level] **then**
17: hash $\leftarrow$ Hash($\mathcal{T}$||salt, node) $\in \{0,1\}^{2\lambda}$
18: $\mathcal{T}$[left_child] $\leftarrow$ hash[: λ]
19: $\mathcal{T}$[right_child] $\leftarrow$ hash[λ :]
20: $\mathcal{T}'$[left_child] $\leftarrow$ True
21: $\mathcal{T}'$[right_child] $\leftarrow$ True
22: **end if**
23: **end for**
24: start_node $\leftarrow$ start_node $+$ NPL[level]
25: **end for**
26: result $\leftarrow$ Leaves($\mathcal{T}$)
27: ok $\leftarrow 0$
28: **for** i $\leftarrow$ pub_nodes **to** Tree_Nodes_To_Store **do**
29: **for** j $\leftarrow 0$ **to** $\frac{\lambda}{8}$ **do**
30: ok $\leftarrow$ ok | Path[i][j]
31: **end for**
32: **end for**
33: **return** result, ok
34: **end if**

```
35: if Variant == Fast then
36:     round_seeds ← ∅
37:     published ← 0
38:     for i ← 0 to t do
39:         if chall_2[i] == True then
40:             round_seeds[i] = Path[published]
41:             published ← published + 1
42:         end if
43:         if chall_2[i] == False then
44:             round_seeds[i] ← ∅
45:         end if
46:     end for
47:     return round_seeds, True
48: end if
```

4.2.2 Merkle trees

The Merkle tree structures use the same `TreeParams` as the seed trees. The key difference between seed and Merkle trees is that we use a bottom-up approach for tree construction. Instead of passing the parent seed to the CSPRNG to generate the children's seed, we concatenate the children's seeds and use this as input to the CSPRNG which outputs the parent seed. The seeds for the Merkle tree are bit strings of size 2λ and the domain separator constant is 32768. Merkle trees require the creation of new functions to process trees from a bottom-up approach.

Building the fast trees also follows a similar approach to the seed trees. First, we compute how many children each node in the second layer has. For each node in this layer, the seed is computed by applying the CSPRNG to the concatenation of the seeds stored in the children of that node. Once the second layer has been computed, the four resulting seeds are concatenated and fed into the CSPRNG, which then generates the root seed. The implementation of `TreeRoot` is shown in Algorithm 5.

Algorithm 5 TreeRoot($\mathtt{cmt}_0$)

Inputs:
$\mathtt{cmt}_0$: data to be stored at each leaf node.
Auxiliary Data:
t: Number of leaves
Variant: Variable that stores the variant of the current instance
λ: Security parameter (in bits)
CSPRNG: Cryptographically secure PRNG
Output:
root_seed: Seed stored at the root node

if variant == small **or** variant == balanced **then**
  $\mathcal{T} \leftarrow$ PlaceOnLeaves($\mathtt{cmt}_0$)
  start_node $\leftarrow$ LSI[0]
  for level $\leftarrow \lceil \log_2(t) \rceil$ **to** 0 **do**
    for i $\leftarrow$ NPL[level] $- 2$ **to** 0 **step** -2 **do**
      left_child $\leftarrow$ start_node + i
      right_child $\leftarrow$ left_child + 1
      parent $\leftarrow$ Parent(left_child)
      $\mathcal{T}$[parent] $\leftarrow$ Hash($\mathcal{T}$[left_child]$\|\mathcal{T}$[right_child])
    end for
    start_node $\leftarrow$ start_node $-$ NPL[level $-$ 1]
  end for
  return $\mathcal{T}[0]$
end if
if variant == fast **then**
  $\mathcal{T}[5 : t + 5] \leftarrow \mathtt{cmt}_0$
  children $\leftarrow [0, 0, 0, 0]$
  if $t \bmod 4 = 0$ **then**
    $\mathtt{children}[i] \leftarrow T/4$ **for all** i
  else if $t \bmod 4 = 1$ **then**
    $\mathtt{children}[0] \leftarrow \lfloor T/4 \rfloor + 1$
    $\mathtt{children}[1..3] \leftarrow \lfloor T/4 \rfloor$
  else if $t \bmod 4 = 2$ **then**
    $\mathtt{children}[0], \mathtt{children}[1] \leftarrow \lfloor T/4 \rfloor + 1$
    $\mathtt{children}[2], \mathtt{children}[3] \leftarrow \lfloor T/4 \rfloor$
  else if $t \bmod 4 = 3$ **then**
    $\mathtt{children}[0], \mathtt{children}[1], \mathtt{children}[2] \leftarrow \lfloor T/4 \rfloor + 1$
    $\mathtt{children}[3] \leftarrow \lfloor T/4 \rfloor$
  end if
  children_offset $\leftarrow 0$
  for i $\leftarrow 0$ **to** 4 **do**
    prep_hash $\leftarrow \emptyset$
    for j $\leftarrow 0$ **to** children[i] **do**
      append $\mathcal{T}[5 + j +$ children_offset$]$ to prep_hash
    end for
    children_offset $\leftarrow$ children_offset + children[i]
    hash $\leftarrow$ Hash(prep_hash)
    $\mathcal{T}[i + 1] \leftarrow$ hash
  end for
  $\mathcal{T}[0] \leftarrow$ Hash($\mathcal{T}[1 : 5]$)
  return $\mathcal{T}[0]$
end if

The second operation that needs to be performed on the Merkle trees is `MerkleProof`. This function computes a proof using the leaves (commitments) and a Boolean challenge vector chall_2. Computing the proof for the fast version is very straightforward, simply run through all the elements in the commitments and check that $\mathsf{chall}_2[i] = \mathsf{true}$, if it is, then we add cmt[i] to the proof. The length of the commitments cmt and the challenge vector chall_2 should be equal.

Computing the proof for the small and balanced version is a bit more involved. First, create a double byte array mtp of size TREE_NODES_TO_STORE, which is a field in the `CROSS` structure. Then the `label_leaves` function is called to create a `flag_tree`, which is a Boolean tree where the leaves are marked true if the corresponding entry $\mathsf{chall}_2[i] = \mathsf{false}$. After computing the Boolean `flag_tree` check for each node that only one of the children is set to true, if that is the case, then add the corresponding seed to `mtp`. If both of them are true, then we do not add anything to the tree in this iteration, but `flag_tree` is updated by setting the parent to `true`. This is the step that enables compression in the proof, since instead of storing both children we would only store the parent in the proof. The implementation of this functionality is shown in Algorithm 6. The ComputeMerkleTree function is the same as TreeRoot except it returns the entire tree instead of just the root node.

Algorithm 6 TREEPROOF($\mathtt{cmt}_0, \mathtt{chall}_2$)

Inputs:
$\mathtt{cmt}_0$: data to be stored at each leaf node.
$\mathtt{chall}_2$: Boolean challenge vector
Auxiliary Data:
t: Number of leaves
Variant: Variable that stores the variant of the current instance
λ: Security parameter (in bits)
CSPRNG: Cryptographically secure PRNG
Output:
mtp: Merkle tree proof

```
1: if variant == small or variant == balanced then
2:     T ← ComputeMerkleTree(cmt_0)
3:     mtp ← ∅
4:     flag_tree ← label_leaves(chall_2)
5:     published ← 0
6:     start_node ← LSI[0]
7:     for level ← ⌈log_2(t)⌉ to 0 do
8:         for i ← NPL[level] − 2 to −1 step −2 do
9:             current_node ← start_node + i
10:            parent_node ← Parent(current_node, level)
11:            flag_tree[parent_node] ← flag_tree[current_node] | flag_tree[current_node + 1]
12:            if flag_tree[current_node] == False and flag_tree[current_node + 1] == True then
13:                mtp[published] ← T[current_node]
14:                published ← published + 1
15:            end if
16:            if flag_tree[current_node] == True and flag_tree[current_node + 1] == False then
17:                mtp[published] ← T[current_node + 1]
18:                published ← published + 1
19:            end if
20:        end for
21:        start_node ← start_node − NPL[level − 1]
22:    end for
23:    return mtp
24: end if
25: if variant == fast then
26:    mtp ← ∅
27:    for i ← 0 to t do
28:        if chall_2[i] == True then
29:            append cmt_0[i] to mtp
30:        end if
31:    end for
32:    return mtp
33: end if
```

The final operation is `RecomputeRoot`, this function takes two double byte arrays cmt_0, proof, and a Boolean challenge vector chall_2 as input. The balanced and small version starts with placing all the entries in cmt_0 on the leaf nodes, then we traverse from leaf nodes to root. In each iteration, a buffer is created to store the data needed to compute the parent; if neither the current node nor its sibling is included in the proof, then simply continue. These nodes are not included due to the compression step, so we know that their parent or ancestor is part of the given proof [5]. If the current node is true, then place the entry in cmt_0 corresponding to the current node in the buffer, else take the corresponding entry in proof and place it in the buffer. The same is then done for the sibling, and finally the buffer is given to the CSPRNG which uses the domain separator constant 32768 and stores the new 2λ bit string in the parent node. The parent is then set to `true`, so we can compute the next level. When the root has been computed, we can run through the remaining part of the proof, since we created the proof to be of length TREE_NODES_TO_STORE, and verify that all the data left is zero bytes if not then the verification fails, finally return the root and whether or not the proof has been modified.

For the fast version, we simply run through all the leaves. If $\mathsf{chall}_2 =$ `true`, then we add the proof entry to the cmt_0 array at index `i` since that index must be empty. This step recreates all the leaf nodes inside the cmt_0 array, then the TreeRoot function can be called, which returns the root node. The implementation of this is shown in Algorithm 7.

Algorithm 7 RECOMPUTEROOT($\mathtt{cmt}_0, \mathtt{mtp}, \mathtt{chall}_2$)

Inputs:
$\mathtt{cmt}_0$: data to be stored at each leaf node.
mtp: Merkle tree proof
$\mathtt{chall}_2$: Boolean challenge vector
Auxiliary Data:
t: Number of leaves
Variant: Variable that stores the variant of the current instance
λ: Security parameter (in bits)
CSPRNG: Cryptographically secure PRNG
Output:
$\mathcal{T}[0]$: seed value stored in root node
ok: Boolean to check if padding has been tampered

```
1: if variant == small or variant == balanced then
2:     T ← place_on_leaves(cmt_0)
3:     flag_tree ← label_leaves(chall_2)
4:     published ← 0
5:     start_node ← LSI[0]
6:     for level ← ⌈log_2(t)⌉ to 0 do
7:         for i ← NPL[level] − 2 to −1 step −2 do
8:             current_node ← start_node + i
9:             parent_node ← Parent(current_node, level)
10:            hash_input ← ∅
11:            if flag_tree[current_node] == False and flag_tree[current_node + 1] == False then
12:                continue
13:            end if
14:            if flag_tree[current_node] == True then
15:                hash_input ← T[current_node]
16:            else if flag_tree[current_node] == False then
17:                hash_input ← T[published]
18:                published ← published + 1
19:            end if
20:            if flag_tree[current_node + 1] == True then
21:                hash_input[2λ :] ← T[current_node + 1]
22:            else if flag_tree[current_node + 1] == False then
23:                hash_input[2λ :] ← T[published]
24:                published ← published + 1
25:            end if
26:            hash ← Hash(hash_input)
27:            T[parent_node] ← hash
28:            flag_tree[parent_node] ← True
29:        end for
30:        start_node ← start_node − NPL[level − 1]
31:    end for
32:    ok ← 0
33:    for i ← published to Tree_Nodes_To_Store do
34:        for j ← 0 to 2λ do
35:            ok ← ok | mtp[i][j]
36:        end for
37:    end for
38:    return T[0], ok == 0
39: end if
```

```
40: if variant == fast then
41:     pub_nodes ← 0
42:     for i ← 0 to t do
43:         if chall_2[i] == True then
44:             cmt_0[i] ← mtp[pub_nodes]
45:             pub_nodes ← pub_nodes + 1
46:         end if
47:     end for
48:     T[0] ← TreeRoot(cmt_0)
49:     return T[0], True
50: end if
```

4.3 CROSS

Now that the building blocks for the signature scheme have been explained, it is time to actually look at the implementation of `KeyGen`, `Sign` and `Verify` for CROSS.

4.3.1 Key Generation

The `KeyGen` function takes no parameters and returns a keypair $(\mathsf{pk}, \mathsf{sk})$. It samples a seed_sk of length 2λ by calling system randomness, which fills the byte array with cryptographically secure random bytes. In the implementation, this is a `rand.read()` function call [22]. Using seed_sk, we can sample (seed_e, seed_pk) by calling the CSPRNG with seed_sk as input and $3 \cdot t + 1$ as the domain separator constant. This call returns a 4λ bit string, which is split into two 2λ bit strings: seed_e and seed_pk. Using seed_pk the `Expand_pk` function can be called, which in the R-SDP case generates $\mathbf{V}$ by calling the CSPRNG which returns a matrix of bytes in $\mathbb{F}_p$.
For R-SDP(G) a matrix $\overline{\mathbf{W}}$ is generated using the CSPRNG that returns a matrix of bytes in $\mathbb{F}_z$. This CSPRNG call also returns the state of the CSPRNG, which needs to be given as input to another CSPRNG call outputs a matrix in $\mathbb{F}_p$. This generates $\mathbf{V}$, so passing the state along is important to have compliant signatures.
`Expand_pk` returns $\mathbf{V}$ for the R-SDP case and it returns $\mathbf{V}, \overline{\mathbf{W}}$ in the R-SDP(G) case. After `Expand_pk`, the implementation diverges for the two cases, for R-SDP a vector $\bar{\mathbf{e}}$ is generated using a CSPRNG which returns a vector in $\mathbb{F}_z$ on input seed_e. The R-SDP case is shown in the pseudo-code as the teal box, and the R-SDP(G) is shown in orange.

In the R-SDP(G) case, we do not immediately generate $\bar{\mathbf{e}}$. First, $\bar{\mathbf{e}}_G$ is generated by calling a CSPRNG which returns a vector of bytes in $\mathbb{F}_z$ on seed_e. After $\bar{\mathbf{e}}$ has been generated, a for-loop is run to compute

e, which is used to generate the syndrome vector **s**. **s** is used as part of the public key. When the values have been generated, the keypair structure will be filled and returned.

The pseudo-code for `KeyGen` is shown in Algorithm 8 and `Expand_pk` is shown in Figure 4.1 for clarity. `Expand_pk` is not shown in the pseudo-code for the `KeyGen` function, but it is the function that generates $\mathbf{V}, \overline{\mathbf{W}}$. For testing purposes a `DummyKeyGen` was also implemented, this was to ensure that the same KeyPair was generated given the same seed_sk. This allowed for comparison with the original C code to ensure that `KeyGen` is compliant with the original CROSS implementation. The only difference between `DummyKeyGen` and `KeyGen` is that `DummyKeyGen` takes seed_sk as input instead of sampling it.

Algorithm 8 KeyGen()

Inputs:
None
Output: sk: Seed_{sk} (secret key seed)
pk: (seed_{pk}, s)
Auxiliary Data:
λ: security parameter;
$g \in \mathbb{F}_p^*$: generator of $\mathbb{E}$;
n: Code length and length of restricted vectors;
k: Code dimension, with $k < n$;
m: Size of the subgroup G is z^m, $m < n$;

// Sampling seeds

1: $\text{seed}_{sk} \xleftarrow{\$} \{0,1\}^{2\lambda}$
2: $(\text{seed}_e, \text{seed}_{pk}) \leftarrow$ CSPRNG-$\{0,1\}^{2\lambda} \times \{0,1\}^{2\lambda}(\text{seed}_{sk}, 3t+1)$
// Sampling random matrices H and $\overline{M}$
3:

$(\overline{\mathbf{W}}, \mathbf{V}) \leftarrow \text{CSPRNG} - \mathbb{F}_p^{m\times(n-m)} \times \mathbb{F}_p^{(n-k)\times k}(\text{seed}_{pk}, 3t+2)$

$\mathbf{V} \leftarrow \text{CSPRNG} - \mathbb{F}_p^{(n-k)\times k}(\text{seed}_{pk}, 3t+2)$

4: $\mathbf{H} \leftarrow [\mathbf{V} \mid \text{Id}_{n-k}]$
// Computing e
5:

$\overline{\mathbf{M}} \leftarrow [\overline{\mathbf{W}} \mid \text{Id}_m]$

$\mathbf{e}_G \leftarrow \text{CSPRNG} - \mathbb{F}_p^m(\text{seed}_e, 3t+3)$

$\bar{\mathbf{e}} \leftarrow \mathbf{e}_G\overline{\mathbf{M}}$

$\bar{\mathbf{e}} \leftarrow \text{CSPRNG} - \mathbb{F}_p^m(\text{seed}_e, 3t+3)$

6: **for** $j \leftarrow 1$ **to** n **do**
7: $\quad \mathbf{e}_j \leftarrow g^{\bar{\mathbf{e}}_j}$
8: **end for**
// Computing the syndrome s
9: $\mathbf{s} \leftarrow \mathbf{e}\mathbf{H}^\top$
// Return secret key sk and public key pk
10: $\text{sk} \leftarrow \text{Seed}_{sk}$
11: $\text{pk} \leftarrow (\text{seed}_{pk}, \mathbf{s})$
12: return (sk, pk);

```
func (c *CROSSInstance[T, P]) Expand_pk(seed_pk []byte) ([]T, []byte) {
        if c.ProtocolData.Variant() == common.VARIANT_RSDP {
                V_tr := c.CSPRNG_fp_mat(seed_pk)
                return V_tr, nil
        } else if c.ProtocolData.Variant() == common.VARIANT_RSDP_G {
                W_mat, state := c.CSPRNG_fz_mat(seed_pk)
                V_tr := c.CSPRNG_fp_mat_prime(state)
                return V_tr, W_mat
        } else {
                return nil, nil
        }
}
```

Figure 4.1: Implementation of `Expand_pk`

4.3.2 Sign

The `Sign` algorithm is given the necessary public data: the security parameter λ, the restriction $\mathbb{E}$, the generator g, weight of the second challenge w, a constant $c = 2t - 1$, and the number of rounds t. The signature returned by the `Sign` function consists of salt, $\text{digest}_{\text{cmt}}$, $\text{digest}_{\text{chall}_2}$, Path, and Proof. In the C implementation, the signature is stored as a byte array, whereas the Go implementation stores the signature in a structure. A helper function can be used to convert the Signature structure into a byte array, mainly used to generate proper KAT data. In the Go implementation, the public data is stored in `CROSS` structure, which can be accessed by the `Sign` method.

The `Sign` function begins by expanding the secret key, producing $(\overline{\mathbf{e}}, \mathbf{H})$ for R-SDP and $(\overline{\mathbf{e}}_G, \overline{\mathbf{e}}, \mathbf{H}, \overline{\mathbf{M}})$ for R-SDP(G) . These values are computed exactly the same way as in the `KeyGen` function, which means that `Expand_sk` is just a subset of the functionality of `KeyGen`. Next, a random seed, $\text{seed} \in \{0,1\}^{\lambda}$, and a salt, $\text{salt} \in \{0,1\}^{2\lambda}$, are sampled. These two values are passed to `SeedLeaves`, which outputs t seeds. When the t seeds have been generated, we enter a for-loop which runs t iterations to simulate the communication between prover and verifier as seen in Figure 3.3.

The for-loop computes the values needed to create the two commitment arrays $\text{cmt}_0, \text{cmt}_1$, both of which contain t entries. The t entries in cmt_0 are given as input to `TreeRoot`, which returns the root seed and stores it in $\text{digest}_{\text{cmt}_0}$. The t cmt_1 entries are given as input to the hash function that returns a bit string of size 2λ, which is stored in $\text{digest}_{\text{cmt}_1}$. Both $\text{digest}_{\text{cmt}_0}$ and $\text{digest}_{\text{cmt}_1}$ are then given as input to a hash function call that stores the resulting bit string of size 2λ in $\text{digest}_{\text{cmt}}$. When the commitment digests have been generated, we can start computing the first challenge used in the 5-pass protocol.

The first challenge is computed using the hashed message $\mathsf{digest}_{\mathsf{msg}}$, which is used to compute $\mathsf{digest}_{\mathsf{chall}_1}$, by calling the hash function with $\mathsf{digest}_{\mathsf{msg}} \| \mathsf{digest}_{\mathsf{cmt}} \| \mathsf{salt}$ as input. The $\mathsf{digest}_{\mathsf{chall}_1}$ is used as input to a CSPRNG which returns a vector in $\mathbb{F}_p$ of size t, and stores the result in chall_1. chall_1 is the second communication step in the CROSS-ID protocol, as seen in Figure 3.3. We then compute the next two communication steps from CROSS-ID, the first is the $\mathsf{digest}_{\mathsf{chall}_2}$ that is named $\mathsf{digest}_{\mathbf{y}}$ in the protocol, this is simply a hash of the t $\mathbf{y}$ entries concatenated together along with $\mathsf{digest}_{\mathsf{chall}_1}$. Then we generate the fixed-weight Boolean chall_2 vector, this is done by calling a CSPRNG function that uses a Fisher-Yates shuffle to ensure that all entries are in $\{0, 1\}$. Finally, we compute Proof and Path, the TreeProof function takes cmt_0 and chall_2 as input and returns the Merkle proof as described in Algorithm 6. The function `SeedPath` takes $\mathsf{seed}, \mathsf{salt}, \mathsf{chall}_2$ as input and returns the path as described in Algorithm 3. Next, the responses are computed, these are the final values communicated in the CROSS-ID.

For testing purposes a function `DummySign` was created, it works like `DummyKeyGen`. It removes the sampling of `root_seed` and `salt`, and instead takes these values as input. In the pseudo-code, the R-SDP specific steps are displayed in teal boxes, while the orange boxes are the steps needed for R-SDP(G) . The pseudo-code for the signature generation is shown in Algorithm 9.

Algorithm 9 Sign(sk, Msg)

Inputs:
sk: secret key $\text{seed}_{sk} \in \{0,1\}^{2\lambda}$
$\text{Msg} \in \{0,1\}^*$
Output: Sgn: signature
Auxiliary Data:
λ: security parameter
$c = 2t - 1$
$g \in \mathbb{F}_p^*$: generator of $\mathbb{E}$
t: number of rounds
w: weight of second challenge

// Expanding secret key

$\overline{\mathbf{e}}, \overline{\mathbf{e}}_G, \mathbf{H}, \overline{\mathbf{M}} \leftarrow \text{ExpandSK}(\text{seed}_{sk})$

$\overline{\mathbf{e}}, \mathbf{H} \leftarrow \text{ExpandSK}(\text{seed}_{sk})$

// Commitment seed generation

$\text{root\_seed} \xleftarrow{\$} \{0,1\}^{\lambda}$, $\text{salt} \xleftarrow{\$} \{0,1\}^{2\lambda}$

$(\text{seed}[1], ..., \text{seed}[t]) \leftarrow \text{SeedLeaves}(\text{root\_seed}, \text{salt})$

// Compute commitments

for $i \leftarrow 1$ **to** t **do**

$\overline{\mathbf{e}}'_G[i], \mathbf{u}'[i] \leftarrow \text{CSPRNG} - \mathbb{F}_z^m \times \mathbb{F}_p^n(\text{seed}[i] \| \text{salt}, i + c)$

$\overline{\mathbf{v}}_G[i] \leftarrow \overline{\mathbf{e}}_G - \overline{\mathbf{e}}'_G[i]$

$\overline{\mathbf{e}}'[i] \leftarrow \overline{\mathbf{e}}'_G[i]\overline{\mathbf{M}}$

$\overline{\mathbf{e}}'[i], \mathbf{u}'[i] \leftarrow \text{CSPRNG} - \mathbb{F}_z^n \times \mathbb{F}_p^n(\text{seed}[i] \| \text{salt}, i + c)$

  $\overline{\mathbf{v}}[i] \leftarrow \overline{\mathbf{e}} - \overline{\mathbf{e}}'[i]$
  for $j \leftarrow 1$ **to** n **do**
    $\mathbf{v}[i]_j \leftarrow g^{\overline{\mathbf{v}}[i]_j}$
  end for
  $\mathbf{u}[i] \leftarrow \mathbf{v}[i] \star \mathbf{u}'[i]$
  $\mathbf{s}'[i] \leftarrow \mathbf{u}[i] \cdot \mathbf{H}^\top$

$\text{cmt}_0[i] \leftarrow \text{Hash}(\mathbf{s}'[i] \| \overline{\mathbf{v}}_G[i] \| \text{salt}, i + c)$

$\text{cmt}_0[i] \leftarrow \text{Hash}(\mathbf{s}'[i] \| \overline{\mathbf{v}}[i] \| \text{salt}, i + c)$

  $\text{cmt}_1[i] \leftarrow \text{Hash}(\text{seed}[i] \| \text{salt}, i + c)$
end for

// Derive final commitment digests

$\text{digest}_{\text{cmt}_0} \leftarrow \text{TreeRoot}(\text{cmt}_0[1], ..., \text{cmt}_0[t])$
$\text{digest}_{\text{cmt}_1} \leftarrow \text{Hash}(\text{cmt}_1[1] \| ... \| \text{cmt}_1[t])$
$\text{digest}_{\text{cmt}} \leftarrow \text{Hash}(\text{digest}_{\text{cmt}_0} \| \text{digest}_{\text{cmt}_1})$

// Computing first challenge

$\text{digest}_{\text{Msg}} \leftarrow \text{Hash}(\text{Msg})$
$\text{digest}_{\text{chall}_1} \leftarrow \text{Hash}(\text{digest}_{\text{Msg}} \| \text{digest}_{\text{cmt}} \| \text{salt})$
$\text{chall}_1 \leftarrow \text{CSPRNG} - (\mathbb{F}_p^*)^t(\text{digest}_{\text{chall}_1}, t + c)$

// Computing first response
for $i \leftarrow 1$ **to** t **do**
  for $j \leftarrow 1$ **to** n **do**
    $\mathbf{e}'[i]_j \leftarrow g^{\bar{e}[i]_j}$
  end for
  $\mathbf{y}[i] \leftarrow \mathbf{u}'[i] + \mathsf{chall}_1[i]\mathbf{e}'[i]$
end for
// Computing second challenge
$\mathsf{digest}_{\mathsf{chall}_2} \leftarrow \mathsf{Hash}(\mathbf{y}[1]\|...\|\mathbf{y}[t]\|\mathsf{digest}_{\mathsf{chall}_1})$
$\mathsf{chall}_2 \leftarrow \mathsf{CSPRNG} - \mathcal{B}_{(t,w)}(\mathsf{digest}_{\mathsf{chall}_2}, t + c + 1)$
// Computing second response
$\mathsf{Proof} \leftarrow \mathsf{TreeProof}(\mathsf{cmt}_0, \mathsf{chall}_2)$
$\mathsf{Path} \leftarrow \mathsf{SeedPath}(\mathsf{seed}, \mathsf{salt}, \mathsf{chall}_2)$
for $i \leftarrow 1$ **to** t **do**
  if $\mathsf{chall}_2[i] = 0$ **then**

$\mathsf{resp}[i]_0 \leftarrow (\mathbf{y}[i], \bar{\mathbf{v}}_G[i])$

$\mathsf{resp}[i]_0 \leftarrow (\mathbf{y}[i], \bar{\mathbf{v}}[i])$

    $\mathsf{resp}[i]_1 \leftarrow \mathsf{cmt}_1[i]$
  end if
end for
// Assembling signature
$\mathsf{Sgn} \leftarrow (\mathsf{salt}, \mathsf{digest}_c\mathsf{mt}, \mathsf{digest}_{\mathsf{chall}_2}, \mathsf{Path}, \mathsf{Proof}, \mathsf{resp})$

return Sgn

4.3.3 Verify

The `Verify` algorithm uses the public data $(\lambda, t, g, \mathbb{E}, w, c = 2t - 1)$, which is stored in the CROSS structure. It takes three parameters: The message `Msg`, the signature `Sgn`, and the public key `pk`. The function returns a Boolean value, depending on whether the signature is valid or not.

The `Verify` function starts exactly the same way as `KeyGen`, expanding the public key. We then generate $\mathsf{digest}_{\mathsf{msg}}$, $\mathsf{digest}_{\mathsf{chall}_1}$, chall_1, and chall_2 the same way as in `Sign`. When these values have been generated, `RebuildLeaves` can be called with $\mathsf{path}, \mathsf{chall}_2, \mathsf{salt}$ as input to build the seed tree. In the for-loop we compute either cmt_0 or cmt_1 depending on the value of $\mathsf{chall}_2[i]$, this is the last step in Figure 3.3 before we choose whether or not to accept. Now, using the values computed in the for-loop, we can compute $\mathsf{digest}_{\mathsf{cmt}_0}$ by calling `RecomputeRoot` with $\mathsf{cmt}_0, \mathsf{proof}, \mathsf{chall}_2$ as input. Then $\mathsf{digest}_{\mathsf{cmt}_1}$ can be computed as the hash of the t cmt_1 entries. The two newly generated

digest values $\mathsf{digest}_{\mathsf{cmt}_0}, \mathsf{digest}_{\mathsf{cmt}_1}$ are concatenated and given as input to a hash function that returns $\mathsf{digest}'_{\mathsf{cmt}}$. Finally, $\mathsf{digest}_{\mathsf{chall}_2}$ is computed using the t values of $\mathbf{y}$ and $\mathsf{digest}_{\mathsf{chall}_1}$. If $\mathsf{digest}_{\mathsf{cmt}} = \mathsf{digest}'_{\mathsf{cmt}}$ and $\mathsf{digest}_{\mathsf{chall}_2} = \mathsf{digest}'_{\mathsf{chall}_2}$ then we return True, otherwise we reject the signature.

In the pseudo-code we distinguish between the R-SDP and R-SDP(G) variants, by using colored boxes. The orange boxes indicate functionality only used by the R-SDP(G) variant, and the teal boxes indicate functionality only used by the R-SDP variant. The implementation of `Verify` is shown in Algorithm 10.

Algorithm 10 Verify(pk, Msg, Sgn)

Inputs:
sk: secret key $\mathsf{seed}_{\mathsf{sk}} \in \{0,1\}^{2\lambda}$
$\mathsf{Msg} \in \{0,1\}^*$
Output: Sgn: signature
Auxiliary Data:
λ: security parameter
$c = 2t - 1$
$g \in \mathbb{F}_p^*$: generator of $\mathbb{E}$
t: number of rounds
w: weight of second challenge

// Recovering public key

$(\overline{\mathbf{W}}, \mathbf{V}) \leftarrow \mathsf{CSPRNG} - \mathbb{F}_z^{m\times(n-m)} \times \mathbb{F}_p^{(n-k)\times k}(\mathsf{seed}_{\mathsf{pk}}, 3t+2)$

$\mathbf{V} \leftarrow \mathsf{CSPRNG} - \mathbb{F}_p^{(n-k)\times k}(\mathsf{seed}_{\mathsf{pk}}, 3t+2)$

$\mathbf{H} \leftarrow [\mathbf{V} \mid \mathsf{Id}_{n-k}]$

$\overline{\mathbf{M}} \leftarrow [\overline{\mathbf{W}} \mid \mathsf{Id}_m]$

// Computing challenges

$\mathsf{digest}_{\mathsf{Msg}} \leftarrow \mathsf{Hash}(\mathsf{Msg})$
$\mathsf{digest}_{\mathsf{chall1}} \leftarrow \mathsf{Hash}(\mathsf{digest}_{\mathsf{Msg}} \| \mathsf{digest}_{\mathsf{cmt}} \| \mathsf{salt})$
$\mathsf{chall}_1 \leftarrow \mathsf{CSPRNG} - (\mathbb{F}_p^*)^t(\mathsf{digest}_{\mathsf{chall1}}, t+c)$
$\mathsf{chall}_2 \leftarrow \mathsf{CSPRNG} - \mathcal{B}_{(t,w)}(\mathsf{digest}_{\mathsf{chall2}}, t+c+1)$

// Computing commitments

$(\mathsf{seed}[i])_{i:\mathsf{chall}_2[i]=1} \leftarrow \mathsf{RebuildLeaves}(\mathsf{Path}, \mathsf{chall}_2, \mathsf{salt})$
for $i \leftarrow 1$ **to** t **do**
  if $\mathsf{chall}_2[i] == 1$ **then**
    $\mathsf{cmt}_1[i] \leftarrow \mathsf{Hash}(\mathsf{seed}[i] \| \mathsf{salt}, i+c)$

$\overline{\mathbf{e}}'_{\mathbf{G}}[i], \mathbf{u}'[i] \leftarrow \mathsf{CSPRNG} - \mathbb{F}_z^m \times \mathbb{F}_p^n(\mathsf{seed}[i] \| \mathsf{salt}, i+c)$

$\overline{\mathbf{e}}'[i] \leftarrow \overline{\mathbf{e}}'_{\mathbf{G}}[i]\overline{\mathbf{M}}$

$\overline{\mathbf{e}}'[i], \mathbf{u}'[i] \leftarrow \mathsf{CSPRNG}{-}\mathbb{F}_z^n \times \mathbb{F}_p^n(\mathsf{seed}[i] \| \mathsf{salt}, i+c)$

    for $j \leftarrow 1$ **to** n **do**
      $\mathbf{e}'[i]_j \leftarrow g^{\overline{\mathbf{e}}[i]_j}$
    end for
    $\mathbf{y}[i] \leftarrow \mathbf{u}'[i] + \mathsf{chall}_1[i] \cdot \mathbf{e}'[i]$
  end if

18: **if** $\mathsf{chall}_2[i] == 0$ **then**
19: $\quad \mathsf{cmt}_1[i] \leftarrow \mathsf{resp}[i]_1$
20:

$(\mathbf{y}[i], \overline{\mathbf{v}}_G[i]) \leftarrow \mathsf{resp}[i]_0$

Check if $\overline{\mathbf{v}}_G[i] \in \mathbb{F}_z^m$

$\overline{\mathbf{v}}[i] \leftarrow \overline{\mathbf{v}}_G[i]\overline{\mathbf{M}}$

$(\mathbf{y}[i], \overline{\mathbf{v}}[i]) \leftarrow \mathsf{resp}[i]_0$

Check if $\overline{\mathbf{v}}[i] \in \mathbb{F}_z^n$

21: $\quad$ **for** $j \leftarrow 1$ **to** n **do**
22: $\quad\quad \mathbf{v}[i]_j \leftarrow g^{\overline{\mathbf{v}}[i]_j}$
23: $\quad$ **end for**
24: $\quad \mathbf{y}'[i] \leftarrow \mathbf{v}[i] \star \mathbf{y}[i]$
25: $\quad \mathbf{s}'[i] \leftarrow \mathbf{y}'[i]\mathbf{H}^\top - \mathsf{chall}_1[1]\mathbf{s}$
26:

$\mathsf{cmt}_0[i] \leftarrow \mathsf{Hash}(\mathbf{s}'[i] \| \overline{\mathbf{v}}_G[i] \| \mathsf{salt}, i + c)$

$\mathsf{cmt}_0[i] \leftarrow \mathsf{Hash}(\mathbf{s}'[i] \| \overline{\mathbf{v}}[i] \| \mathsf{salt}, i + c)$

27: $\quad$ **end if**
28: **end for**
// **Computing challenges**
29: $\mathsf{digest}_{\mathsf{cmt}_0} \leftarrow \mathsf{RecomputeRoot}(\mathsf{cmt}_0, \mathsf{Proof}, \mathsf{chall}_2)$
30: $\mathsf{digest}_{\mathsf{cmt}_1} \leftarrow \mathsf{Hash}(\mathsf{cmt}_1[1] \| ... \| \mathsf{cmt}_1[t])$
31: $\mathsf{digest}'_{\mathsf{cmt}} \leftarrow \mathsf{Hash}(\mathsf{digest}_{\mathsf{cmt}_0} \| \mathsf{digest}_{\mathsf{cmt}_1})$
32: $\mathsf{digest}'_{\mathsf{chall}_2} \leftarrow \mathsf{Hash}(\mathbf{y}[1] \| ... \| \mathbf{y}[t] \| \mathsf{digest}_{\mathsf{chall}_1})$
33: **if** $\mathsf{digest}_{\mathsf{cmt}} == \mathsf{digest}'_{\mathsf{cmt}}$ **and** $\mathsf{digest}_{\mathsf{chall}_2} == \mathsf{digest}'_{\mathsf{chall}_2}$ **then**
34: $\quad$ **return** True
35: **end if**
36: **return** False

4.4 IMPLEMENTATION CHALLENGES AND INSIGHTS

During our implementation of CROSS, we encountered some challenges and learned some valuable insights. We found that directly implementing from the specification and pseudo-code was not enough to get matching signatures due to differences between the reference code and pseudo-code. Some of these differences were given as feedback to the specification authors, as noted in Section 5.7.

We did not have any problems with the bit packing, despite it seeming non-trivial to implement. It turned out to be very straightforward, since the syntax for bit operations in C exactly matches the syntax for bit operations in Go. This meant that we only needed to handle the conditional statements, which is trivial. This is also the reason why there are no direct test cases for pack/unpack.

One of the more complicated parts to implement was the tree structures. Despite binary trees usually being well supported by various programming languages and therefore easy to implement, the heavily unbalanced nature of the trees made implementation and debugging problematic, as the size of the trees made getting an overview of the effects non-trivial. The trees were represented as an array with direct indexing, which lessens the load of having to manage pointers and relations between nodes but can make small index errors have massive effects on the results. These issues are also part of the reason why some other code-based schemes, such as MEDS, chose not to use unbalanced trees, instead relying on balanced trees but not using all generated leaf nodes [36].

An error we encountered multiple times was the unclear behavior of the built-in `append` function on slices. This led to an error where the array we were slicing would get updated with the values we tried to append onto the slice. This was problematic for generating hashes, where we wanted to append the salt to the byte array before hashing. This would overwrite the next value in the tree with the salt. In Function 4.6, we have shown this behavior and what the impact is.

Listing 4.6: `append` behavior when using slices

```
1 func main() {
2         x := []byte{1, 2, 3, 4, 5}
3         slice := x[:3]
4         fmt.Println("slice: ", slice) // slice:  [1 2 3]
5         slice = append(slice, 6)
6         fmt.Println("slice: ", slice) // slice:  [1 2 3 6]
7         fmt.Println("x: ", x) // x:  [1 2 3 6 5]
8 }
```

Part III

REFLECTION

In this part, we will evaluate the implementation. Focus is placed on the tests used throughout the development phase to ensure compatibility with signatures from the reference code, as well as experimental results for performance. Performance is measured by time and memory usage, and arguments for our implementation being constant-time are presented through theoretical and experimental means. Finally, we go through the feedback given to the CROSS team on the specification and present our conclusions on the project.

5 EVALUATION

5.1 TEST SUITE

Our test suite for CROSS was designed to validate the correctness of the individual components used in CROSS, as well as the overall functionality of the scheme. The tests were performed on all variants with deviating behavior. We implemented unit tests for both tree variants, the hash function, and the CSPRNG, which is based on SHAKE. To verify that our Go implementation matches the expected behavior, we derived the test vectors from the C reference implementation. This allowed us to ensure that for any given input, all components produced outputs that were equal to the C implementation.

In addition to unit tests, we validated the core operations of the CROSS scheme: key generation, signing, and verification. To be able to properly test the functions that sample from system randomness, we needed to create a deterministic setup. We implemented two extra functions `DummyKeyGen` and `DummySign`; these functions take randomness as input, instead of sampling it from system randomness. This allowed us to print the values sampled in the C implementation and use them as input for the dummy functions. This allowed us to verify the keys generated from `KeyGen` and the signature generated from `Sign`. `Verify`, did not need a dummy function since no randomness is being sampled, all the data needed is in the signature. The test values in the C code were generated by creating a `main.c` file, which imported the `CROSS` library. This new C file simply called the necessary functions in the `CROSS` library, and printed the required values. In some places, we modified the library to print internal values of these functions.

We also implemented negative tests, to see if the `Verify` algorithm, would accept an invalid tampered signature. This was done by generating a valid signature and flipping the first bit and confirming that the verification failed, as expected. This process was repeated for each bit in the signature. After each bit flip, the signature was reverted to the original, so only one bit was flipped at a time. This approach was known to find errors among some signature schemes in the LibOQS collection of Post-Quantum schemes [19]. Doing this test guarantees that our implementation indeed does work on every part of the signature as expected, since the specification does not note any redundant bits [5]. We considered these tests sufficient to validate the

correctness of our implementation, as the purpose of testing was not to verify the security of the signature scheme itself.

5.2 KNOWN ANSWER TESTS

Known Answer Tests (KAT) are commonly used in the NIST cryptographic proposal process. Specifically, for the Post-Quantum Signature Schemes call, the KAT framework consists of paired request and response files. Each request file provides fixed inputs and seeds, while the corresponding response file contains the generated outputs. The use of KAT files makes it easier to test other implementations of the same algorithm, as the new implementation should output exactly the same response file given the request file. If discrepancies are found, it would usually be a sign of an implementation error or deviation from the specification [32].

During the creation of NIST KAT requests and responses, there is no use of randomness from any actual random sources. Instead, seeded randomness is generated by using the NIST-provided PRNG based on `AES_CTR_DRBG` (as specified in SP800-90A section 10.2.1.5.1) [31]. This ensures that differences caused by randomness or differing PRNG implementations would not cause any different outputs.

The NIST provided PRNG does not exist in Go, so to avoid having to re-implement it, we simply adjusted the KAT generation script to also output the randomness from the NIST API's RNG calls. We then utilized our dummy functions for key generation and signing, as specified in Section 5.1, where we could provide randomness directly as a parameter rather than fetching from the system randomness. Our implementation passed the KATs when utilizing this adjusted generation script on the reference code and our Go implementation.
In CROSS, there are KAT pairs for each variant and security level, which gives us 18 different KAT pairs. Each KAT pair contains 100 request/response pairs where the random values are different and the message length increases by 33 for each request. This ensures that we cover all variations of the scheme.

Passing these KAT tests along with our own test suite makes us confident that we are compliant with the specification and reference code. Directly using signatures between C and Go is possible, the only step needed is to write a parser that takes the byte array generated by the C implementation, and turn it into a byte array in Go, this can then be converted into the signature structure using the `ToSig` function, which is defined on the CROSS instance. A signature generated in Go can be verified by the C implementation by converting the signature

structure into a byte array using `ToBytes`, and then passing this byte array along to the C implementation. This means signatures generated in C can be verified using the Go implementation, and vice versa, assuming that a parser is written to get it into the expected format.

5.3 EXPERIMENTS

Our experimental setup was a dedicated server running Arch Linux 6.12.1-arch1-1. The server has Go version 1.24.3 linux/amd64 and the CROSS C implementation was compiled using gcc version 15.1.1 20250425 (GCC). The server has 67 GB of RAM available (as per `free -h`) and an Intel i7-7700 processor running at 3.60 GHz with Turbo Boost disabled. The processor supports AVX2, which is used by the optimized version of the C implementation.

AVX2 is a set of CPU instructions that support performing the same operation on multiple different pieces of data at the same time, a process commonly known as SIMD - Single Instruction, Multiple Data. This can provide significant speed-ups if used correctly as it allows parallelization of computation which is useful for operations like matrix multiplication [24]. During parts of CROSS, we have large matrix operations that can utilize this well. We manually compiled the libkeccak library for AVX2 from the XKCP library, which contains the SHAKE hashing algorithm. This is used in the C implementation on our server whereas the Go implementation utilized the SHAKE function from the `x/crypto` package.

We chose to run the experiments for two metrics, time and memory use. Both of these impact performance and therefore the usability of the scheme. This is especially important in embedded systems, where there are strict constraints on the available computation and memory space. Key and signature sizes were not measured, as they are already covered in the CROSS specification and are part of the NIST evaluation process, both are shown in Table 3.1. Although the CROSS reference implementation provides clock cycle measurements from their own benchmarking, we cannot directly compare these results, due to hardware differences. Instead, we measured both our Go implementation and the reference C implementation on our hardware setup.

5.3.1 Benchmarking - Time

One of the most important factors when it comes to the use of cryptographic algorithms is time. Therefore, it is an important part of the NIST evaluation process. We initially made our CROSS implementa-

tion with a full focus on correctness and full adherence to the provided KAT files, only looking to optimize after this was done. Although we did expect a slightly worse performance time-wise due to implementing in Go, we found that the code is roughly 6-7 times slower than the C implementation. We tried benchmarking the Go code, the reference C code, and the optimized C code to see how they compared to each other. It is important to note that all the C code was compiled using `gcc -O3`, which allows `GCC` to perform many optimizations, however, these optimizations could potentially lead to bugs or other issues if the original code has any undefined behavior or relies on a specific order of operations [44]. The runtime graph for the three implementations is shown in Figure 5.1.

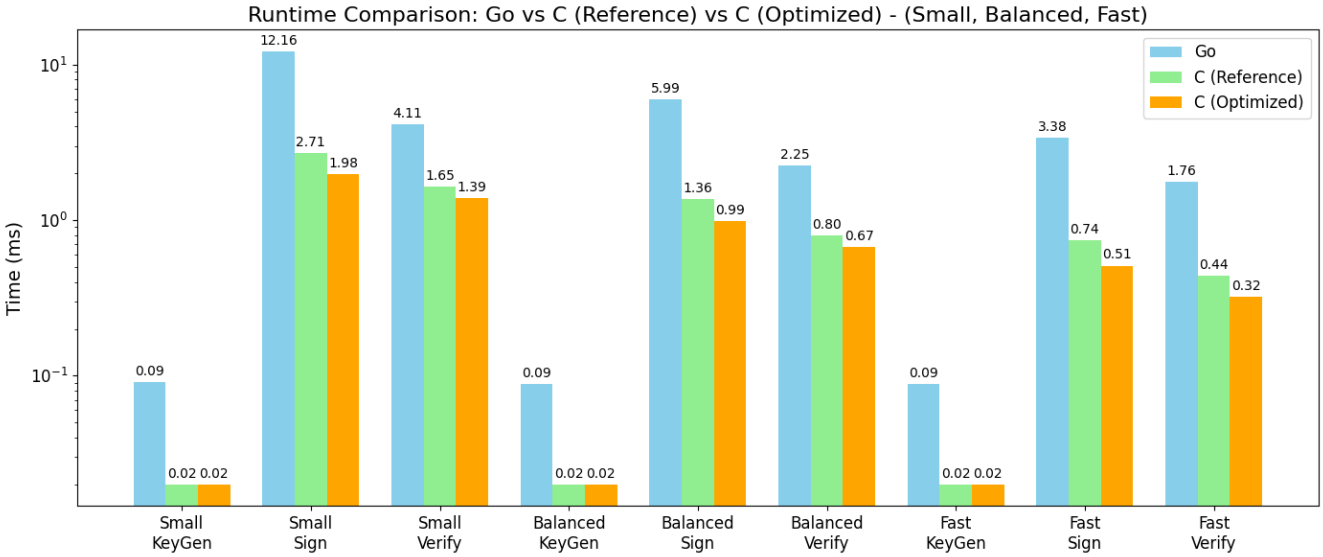

Figure 5.1: Runtime comparison between the implementation of Go, reference C, and optimized C

Due to the difference in runtime performance, we performed profiling utilizing the internal Go timing tools on our own computers. Although these add an overhead when analyzing and therefore cannot be trusted for regular time results, they do provide some insight into the bottlenecks in the code, and allowed us to identify the main bottlenecks for the Go implementation. In Figure 5.2, it is shown how the profiling tool displays the amount of time spent at each line in the `Sign` function. We can see that most of the time is spent in CSPRNG calls or in the matrix calculation `Fp_vec_by_fp_matrix` on line 85.

```
  10ms      1.49s (flat, cum) 95.51% of Total
     .          .     43:func (c *CROSSInstance[T, P]) Sign(sk, msg []byte) (Signature, error) {
     .          .     44:   signature := Signature{}
     .       10ms     45:   V_tr, W_mat, e_G_bar, e_bar := c.Expand_sk(sk)
     .          .     46:   salt := make([]byte, 2*c.ProtocolData.Lambda/8)
     .          .     47:   root_seed := make([]byte, c.ProtocolData.Lambda/8)
     .          .     48:   rand.Read(salt)
     .          .     49:   rand.Read(root_seed)
     .          .     50:   signature.Salt = salt
     .       40ms     51:   round_seeds := c.SeedLeaves(root_seed, salt)
     .          .     52:   e_bar_prime := make([]byte, c.ProtocolData.T*c.ProtocolData.N)
     .          .     53:   v_bar := make([]byte, c.ProtocolData.T*c.ProtocolData.N)
     .          .     54:   u_prime := make([]T, c.ProtocolData.T*c.ProtocolData.N)
     .          .     55:   s_prime := make([]T, c.ProtocolData.N-c.ProtocolData.K)
     .          .     56:   cmt_0 := make([][]byte, c.ProtocolData.T)
     .          .     57:   cmt_1 := make([]byte, c.ProtocolData.T*((2*c.ProtocolData.Lambda)/8))
     .          .     58:   v_G_bar := make([]byte, c.ProtocolData.T*c.ProtocolData.M)
     .          .     59:   for i := 0; i < c.ProtocolData.T; i++ {
     .          .     60:           var csprng_input []byte
     .          .     61:           csprng_input = append(append(csprng_input, round_seeds[i]...), salt...)
     .          .     62:           dsc := uint16(0 + i + (2*c.ProtocolData.T - 1))
     .       10ms     63:           round_state := c.CSPRNG_init(csprng_input, dsc)
     .          .     64:           if c.ProtocolData.Variant() == common.VARIANT_RSDP {
     .       70ms     65:                   e_bar_prime_i, state := c.CSPRNG_fz_vec_prime(round_state)
     .          .     66:                   round_state = state
     .          .     67:                   copy(e_bar_prime[i*c.ProtocolData.N:(i+1)*c.ProtocolData.N], e_bar_prime_i)
     .          .     68:           } else {
     .          .     69:                   e_G_bar_prime, state := c.CSPRNG_fz_inf_w_prime(round_state)
     .          .     70:                   round_state = state
     .          .     71:                   v_G_val := c.Fz_vec_sub_m(e_G_bar, e_G_bar_prime)
     .          .     72:                   v_G_val = c.Fz_dz_norm_m(v_G_val)
     .          .     73:                   copy(v_G_bar[i*c.ProtocolData.M:(i+1)*c.ProtocolData.M], v_G_val)
     .          .     74:                   e_bar_prime_i := c.Fz_inf_w_by_fz_matrix(e_G_bar_prime, W_mat)
     .          .     75:                   e_bar_prime_i = c.Fz_dz_norm_n(e_bar_prime_i)
     .          .     76:                   copy(e_bar_prime[i*c.ProtocolData.N:(i+1)*c.ProtocolData.N], e_bar_prime_i)
     .          .     77:           }
     .       30ms     78:           v_bar_i := c.Fz_vec_sub_n(e_bar, e_bar_prime[i*c.ProtocolData.N:(i+1)*c.ProtocolData.N])
     .       40ms     79:           v := c.Convert_restr_vec_to_fp(v_bar_i)
     .       50ms     80:           v_bar_i = c.Fz_dz_norm_n(v_bar_i)
     .          .     81:           copy(v_bar[i*c.ProtocolData.N:(i+1)*c.ProtocolData.N], v_bar_i)
     .       70ms     82:           u_prime_i := c.CSPRNG_fp_vec_prime(round_state)
     .          .     83:           copy(u_prime[i*c.ProtocolData.N:(i+1)*c.ProtocolData.N], u_prime_i)
     .       50ms     84:           u := c.Fp_vec_by_fp_vec_pointwise(v, u_prime_i)
     .      680ms     85:           s_prime = c.Fp_vec_by_fp_matrix(u, V_tr)
     .          .     86:           s_prime = c.Fp_dz_norm_synd(s_prime)
     .          .     87:           var cmt_0_i_input []byte
```

Figure 5.2: Runtime profiling of the `Sign` algorithm

5.3.2 Benchmarking - Memory

Measuring memory usage across C and Go implementations in the same way is a non-trivial task. Therefore we had to measure the memory usage in two different ways, which might introduce some inconsistencies.
In Go, we utilized the built-in benchmarking features to measure the number of bytes allocated on average during runs. Here we specifically focused on heap allocations since most, if not all, of the variables were heap-allocated, likely due to the variables being used outside of the scope of the function. Specifically, the two types of trees are large allocations of memory.

In C we measured memory usage utilizing GCC flags "-g" and "-fstack-usage" as the C implementation did not utilize any direct allocations

which would be based in the heap, therefore we had to measure stack size instead. The stack-usage flag creates `.su` files which provide the compiler's estimate for most possible stack usage for each function in a file. The file also notes whether it could find a suitable bound or whether the stack usage is potentially unbounded. Generally, we found that there were no large differences between the reference implementation and the optimized implementation in terms of the memory used. In Figure 5.3 the `CROSS.c.su` file is shown for the C reference implementation.

```
1 /home/cross/CROSS_submission_memory/Reference_Implementation/lib/CROSS.c
    :132:6:CROSS_keygen     8064    static
                                                                /home/cross
    /CROSS_submission_memory/Reference_Implementation/lib/CROSS.c:182:6:
    CROSS_sign       209536  static
                                                                /home/cross
    /CROSS_submission_memory/Reference_Implementation/lib/CROSS.c:379:5:
    CROSS_verify     96112   dynamic,bounded
```

Figure 5.3: Contents of CROSS.c.su for R-SDP-1-Balanced

The Go implementation has been compared to the C implementation in terms of memory and is shown in Figure 5.4. It is clear that the Go implementation uses 1.75 times as much memory for key generation compared to C, and Go is roughly 7 times as expensive in terms of memory for `Sign` and `Verify`. When calculating the path and proof for our trees, we compute our trees again in a new variable, and since Go's garbage collector might not run as often, this could cause multiple trees to be in memory at the same time. Trees are one of the largest allocations in our code, and this would therefore have a significant impact on memory measurements.

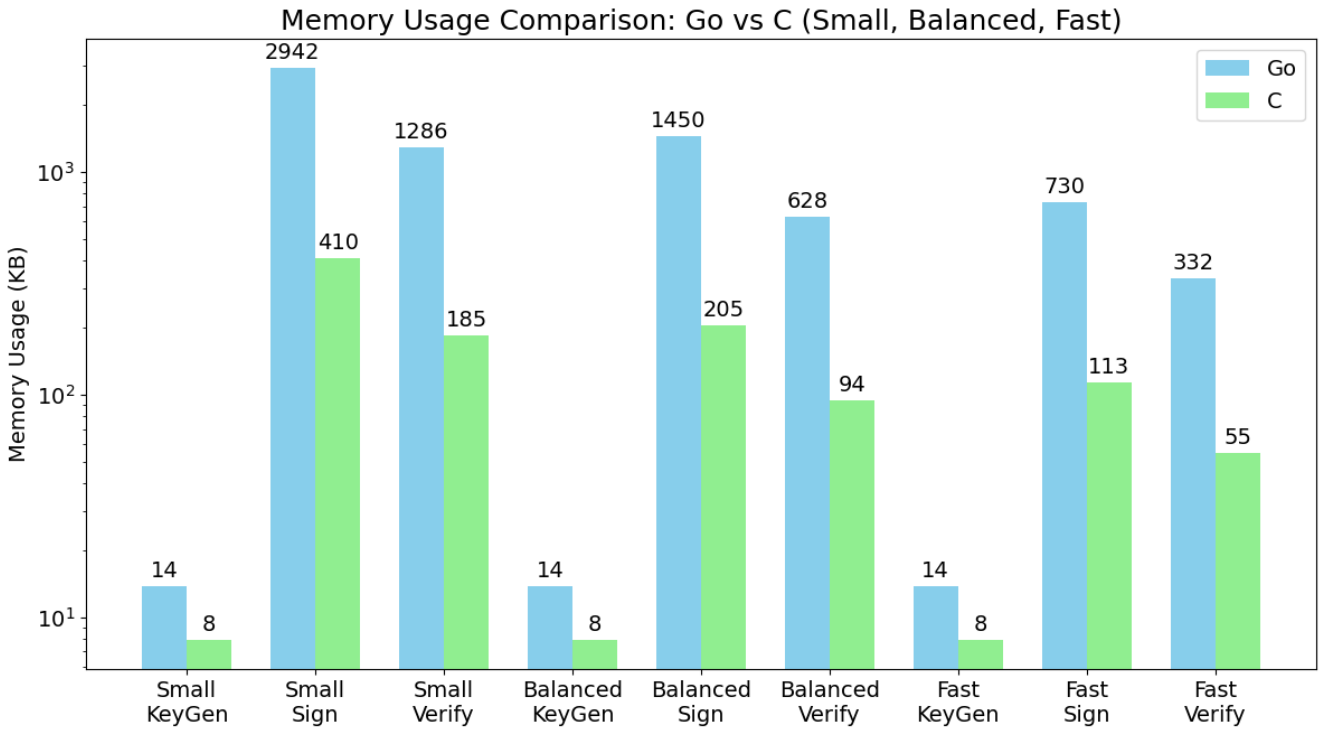

Figure 5.4: Memory usage comparison between Go and C

5.4 CONSTANT-TIME IMPLEMENTATION – `dudect`

Constant-time implementations are important for cryptographic systems, as they prevent side-channel attacks on timing. If an implementation's execution time depends on secret values, it might be possible for an attacker measuring time to leak parts or all of the secret values. This is opposed to non-cryptographic programs that often benefit from variable-time optimizations as they do not handle secret data. Although these optimizations can improve performance, they should be avoided in cryptographic code as they might introduce an attack vector for timing attacks. We will focus on the CROSS signing operation in this part, as verification does not use any secret inputs, and key generation consists of sampling into fixed-length arrays and a few vector/matrix operations. In order to determine whether our implementation is constant-time, we can both reason theoretically about the aspects of our code and perform experimental testing with a tool called `dudect`.

5.4.1 Theoretical constant-time

On a theoretical level, it could be argued whether the code has any branching or memory access patterns based on secrets. Of course, arguing on a source code level in a compiled language like Go is not ideal, since the compiler might change some operations to optimize the resulting binary. However, checking a full compiled program of this size is not feasible in a reasonable amount of time. This problem is common and is one of the reasons behind tools such as `dudect` to help test constant-time implementations [40].

We have one input that can be considered secret - namely the secret key, the randomness obtained at the start of the sign operation root_seed should be deemed secret. If the root_seed becomes known to an attacker, they would be able to generate the entire seed tree and therefore all the randomness used in the sign protocol.
If we relate this to the underlying zero-knowledge ID protocol "CROSS-ID" from section 3.2, a leak of root_seed would essentially be equivalent to always giving the response for $\mathsf{chall}_2 = \mathsf{true}$ to the attacker. In CROSS-ID, the secret is $\mathbf{e}$ and the randomness helps hide this secret. If the attacker already knows the seed, they could then request $\mathsf{chall}_2 = \mathsf{false}$ and receive the pair $(\mathbf{y}, \mathbf{v})$. $\mathbf{v} = \mathbf{e} * (\mathbf{e}')^{-1}$, however, since $\mathbf{e}'$ is generated using the seed, an attacker can now find the secret $\mathbf{e}$. This is typical in zero knowledge protocols where an attacker has two responses to a binary challenge from the same protocol execution.

The secret key is expanded to four variables at the beginning of signing. Two of these, $\mathbf{V}$ and $\overline{\mathbf{W}}$, can be constructed from the public key, so they cannot be considered secret. $\overline{\mathbf{e}}_G$ and $\overline{\mathbf{e}}$ are both deemed secret and used in a series of vector and matrix calculations. All the calculations happen in a group structure and, therefore, are done by calls to functions. It would be reasonable to assume that these functions are constant-time as they are implemented without branching or similar optimizations, however, it will depend on the compiler. Even if the compiler introduces a small optimization, it would be hard for an attacker to exploit due to the number of elements that are being worked on and all of the elements originate from a previous CSPRNG output. The results of these calculations are also only revealed when $\mathsf{chall}_2 = \mathsf{false}$.

root_seed is only used in two operations, both of which relate to the seed tree. Once for creating the leaves, and once for calculating the path. The path is a part of the signature and, therefore, public. Therefore, all that remains is the use of the root_seed as an input to the SHAKE function for building the seed tree. We can assume SHAKE to be safe for cryptographic use when input size is known as seen in Section 2.3. Since the root_seed size is determined by the scheme

variant and security level, it would be public knowledge and therefore, SHAKE operates on root_seed in a constant-time manner. Since there is no branching or checks on the root_seed, we can state that it does not add variation to the runtime.

5.4.2 Experiments with dudect

Experimentally determining whether a program runs in constant-time is somewhat of a challenge, yet measurements can help to determine if there are any timing differences [40]. A tool named `dudect` has been proposed to measure and perform statistical analysis on timing differences, unlike other tools that are dependent on modeling the CPU or other hardware parts to do static analysis [40]. Although `dudect` was created in C, various open-source re-implementations in other languages exist. We chose to utilize a Go implementation by Yolan Romailler called `go-dudect` [42]. Unlike the `dudect` paper, which has a t threshold of 4.5 [40], the t threshold value in the Go implementation is set to 5 but can be adjusted freely from the variable. As none of our measurements were between 4.5 and 5, this had no impact at all.

We ran our tests with `dudect` on a dedicated server with no other non-system processes running, which should minimize the noise in the measurements. This is the same experimental server setup that is used for benchmarking in 5.3. We tested all CROSS variants in our implementation on security level 1, higher security levels did not show any differences, when running `dudect` for a period of 3 hours. We made a setup of fixed-vs-random, either signing a message "Hello World!" or a message of 12 random bytes. Table 5.1 shows the results of these tests: Measurements are in millions, max t is the threshold value, and $\left(\frac{5}{\tau}\right)^2$ is how many measurements it would take to detect the leak, if present [42].

Variant	**Measurements (M)**	**max t**	$\left(\frac{5}{\tau}\right)^2$
R-SDP 1 Balanced	1.63	+1.52	2.87×10^{13}
R-SDP 1 Small	0.78	+1.10	1.24×10^{13}
R-SDP 1 Fast	2.27	+2.47	2.11×10^{13}
R-SDP(G) 1 Balanced	2.42	+1.43	7.14×10^{13}
R-SDP(G) 1 Small	0.94	+1.86	6.43×10^{12}
R-SDP(G) 1 Fast	4.85	+1.49	2.63×10^{14}

Table 5.1: Results from running the CROSS variants with `dudect`

The immediate conclusion from `dudect` was as expected from the earlier discussion of theoretical constant-time in Section 5.4.1. Our `Sign` function appears to run in constant-time no matter which message is

signed, as long as the length stays the same. As the t threshold value for constant-time in `dudect` measurements is 4.5, our implementation's measurements are well below this.

We also performed a series of longer measurements, however, due to other processes not having been disabled completely throughout the measurements, the results might have been slightly affected. Yet, they should still have some value since the statistical analysis would remove most outliers. It could also be reasonable to assume that the other processes did not cause any large spikes of activity, instead they behaved as a stable workload in the background. The t value moved towards 1.24 as measurements increased to 6.5M for R-SDP 1 Balanced. A series of smaller scale `dudect` tests for signing the same message under different keys also showed results close to table 5.1.

Finally, we performed tests with different message lengths. Here, we found signs of variable-time behavior as the signing time varied based on message length. With a setup of either the fixed "Hello World!" message or a random message of up to 5000 bytes, we got a max t value of 59.64 over a series of 1.35M measurements, this is well above the threshold of 4.54. This result makes sense from a technical point of view, since the SHAKE function used takes more time to absorb a larger input, in this case the message. Since the message is publicly known, the length is not a secret and can therefore be allowed to affect the runtime of the signing algorithm without problems. It should be noted that `dudect` cannot solely be trusted for determining constant-time [40], however, combined with our theoretical analysis, we can reasonably claim that our implementation does not have any obvious timing vulnerabilities.

5.5 OVERALL COMPARISON BETWEEN C AND GO IMPLEMENTATIONS

Generally, the C implementation was faster than our Go implementation, which is to be expected. It should be noted that part of this speed difference can be attributed to the use of AVX2 in C. Using AVX2 or similar in Go is supported, but would require significant efforts to integrate in our implementation. However, even without that optimization, the Go implementation brings several advantages on its own. Go packages are incredibly easy to import and use. This can be used to guarantee that a developer does not accidentally perform actions during import and compile that would lead to slowdowns or misconfigurations. Both misconfigurations and cryptographic failures are identified as significant security risks in the OWASP Top 10 list of web application security concerns [37]. Go is also good at easily

making cross-compilation to other platforms possible as it is natively supported; this is unlike C where additional steps need to be performed.

An advantage of the Go implementation is its strong type safety, which ensures that operations are only permitted on compatible types and prevents misuse of data in unintended contexts. By packaging the implementation as a Go library that exposes all the necessary components, developers can easily import and use it in their own projects. This makes the CROSS signature scheme available for projects that are suitable for Go, such as web services and large-scale distributed networks. This also has the potential to make it easy to switch from a previous cryptographic library which follows the same simple style.

5.6 FUTURE IMPROVEMENTS

One of the main performance improvements possible would be changes to the bottlenecks in the code, particularly the CSPRNG and Hash functions, as well as some of the matrix calculations, as noted in 5.3.1. These could be optimized by coding directly in Go's assembly, AVX instructions, or by importing the corresponding C functions as a library. Importing the C functions is not trivial, though, as the entire CROSS implementation has high coupling of its internal functions.

Another improvement that would be interesting to investigate and implement would be a more direct parallelization of the t rounds through the use of threading. However, to avoid performance impact by using too many threads, careful consideration of how to handle it best would be needed.

Finally, something that could improve the memory measurements and performance for our Go implementation would be different management of tree structures in memory. This could be done by saving the tree after it has been computed by `SeedLeaves` or `TreeRoot`, and then giving this tree as a parameter when the path and proof are computed. Due to multiple functions using parts of the trees, we could avoid building the same tree twice. Although this would keep the tree in memory for a longer period of time during signing, it would avoid new allocations and also provide a general speedup.

5.7 SPECIFICATION FEEDBACK

During the development of our CROSS implementation, we identified several errors and inconsistencies in the CROSS specification (version 2.1), mainly in the included pseudo-code. We noticed that there were inconsistencies between the pseudo-code and the C implementation. This caused some issues during our first implementation from the pseudo-code itself and led to parts being re-implemented based on the C implementation instead. We reached out to the CROSS team to share our observations and received confirmation on the errors we pointed out. They noted that the specification will be revised with our corrections for the next update. Listed below are the errors and inconsistencies that were still present in version 2.1.

- Page 35 - Algorithm 9 - Line 3: Startnode should be 1 (as in the implementation).
- Page 35 - Algorithm 9 - Line 3: Path should not be set to the empty set since it is provided to the function as input and it is later referenced into.
- Page 37 - algorithm 13 - line 2 (and page 34 - algorithm 7 - line 2): ComputeNodesToPublish / $\mathsf{Label}_{\mathsf{leaves}}$ have inverted logic between the two algorithms in the reference code. Even though the pseudo-code makes it appear, as if they should behave the same way, since they both call "ComputeNodesToPublish". Maybe add a parameter to that function call with the relevant boolean to indicate which value the challenge proof value should be or distinguish between the two functions in the pseudo-code.
- Page 37 - Algorithm 13 - Line 7: Parent is never used in the pseudo-code. This is due to the pseudo-code lacking a part of the implementation that ensures proper changes to T′. Pseudo-code should specify something for this line from the reference code: "$\mathsf{flag}_{\mathsf{tree}}[\mathsf{parent}_{\mathsf{node}}] = (\mathsf{flag}_{\mathsf{tree}}[\mathsf{current}_{\mathsf{node}}] == \mathsf{COMPUTED}) \| (\mathsf{flag}_{\mathsf{tree}}[\mathsf{SIBLING}(\mathsf{current}_{\mathsf{node}})] == \mathsf{COMPUTED});$"
- Page 38 - algorithm 15 - line 8: T is used instead of T′ (as in the implementation).
- Not an error, but something that can cause confusion is that the domain separator constants are being concatenated to the input of the CSPRNG, it would probably be more clear if this was shown to be two different parameters to the CSPRNG call, like it is done in the reference code.

6 CONCLUSION

This thesis has served both as an introduction to fundamental concepts and constructions utilized in many post-quantum cryptographic schemes and as a detailed exploration of the CROSS signature scheme, including a complete implementation in the Go programming language. It explains important concepts such as seed trees and Merkle trees, as well as basic interactive protocols and the transformation to non-interactive protocols using Fiat-Shamir.

The Go implementation only relies on Google's SHA3 library. This was done to avoid using libraries that could become outdated or insecure, and also to prevent supply chain attacks. Although the C implementation is faster and uses less memory, the Go implementation offers some distinct advantages, such as easier integration for other Go-based projects and improved safety features inherent to the language. Our Go implementation is also constant-time, based on both a theoretical evaluation and practical experiments using `dudect`. Possible future optimizations for the Go implementation have been identified and may significantly reduce the current performance gap with the C version.

To the best of our knowledge, this is the first publicly available implementation of CROSS in Go. Given the ongoing NIST standardization process, this contribution may serve as a foundation for future Go-based post-quantum cryptographic libraries, if CROSS becomes standardized. The Go and C implementations are compatible, which was verified by our test cases and the NIST submitted Known Answer Tests (KAT). A signature generated by either the C or Go implementation can be verified by the other implementation, given that the signature is formatted correctly.

During the development of our implementation, we identified and reported several errors and ambiguities in the CROSS specification that only became apparent when addressing the practical details of the implementation. By providing this feedback, we have contributed to improving the clarity of the specification and supporting the development of future implementations.

ACKNOWLEDGMENTS

We are extremely grateful to *Diego F. Aranha* for supervising our project and providing valuable feedback and material.

We are also grateful to *Christian Amstrup Petersen* who helped with structural advice for our Go implementation.

Thanks to the *CROSS team* for considering our feedback on the specification and incorporating it into a future version.

Finally, we extend our thanks to both *Niels Olof Bouvin* and *André Miede*, both of whom worked on the LaTeX template used to design and format our thesis.

BIBLIOGRAPHY

[1] Gorjan Alagic et al. *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process (NIST IR 8413–upd1)*. NIST Interagency or Internal Report 8413–upd1. National Institute of Standards and Technology, July 2022. DOI: 10.6028/NIST.IR.8413-upd1. URL: https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8413-upd1.pdf.

[2] Nicolas Aragon, Julien Lavauzelle, and Matthieu Lequesne. *decodingchallenge.org*. Accessed: 2025-06-08. URL: https://decodingchallenge.org/home/documentation.

[3] Marco Baldi, Massimo Battaglioni, Franco Chiaraluce, Anna-Lena Horlemann-Trautmann, Edoardo Persichetti, Paolo Santini, and Violetta Weger. *A New Path to Code-based Signatures via Identification Schemes with Restricted Errors*. 2021. arXiv: 2008.06403 [cs.CR]. URL: https://arxiv.org/abs/2008.06403.

[4] Marco Baldi, Sebastian Bitzer, Alessio Pavoni, Paolo Santini, Antonia Wachter-Zeh, and Violetta Weger. *Generic Decoding of Restricted Errors*. 2023. arXiv: 2303.08882 [cs.CR]. URL: https://arxiv.org/abs/2303.08882.

[5] Marco Baldi et al. *Codes and Restricted Objects Signature Scheme (CROSS) Specification*. Version 2.1. Accessed: 2025-06-06. 2025. URL: https://www.cross-crypto.com/CROSS_Specification_v2.1.pdf.

[6] Michele Battagliola, Riccardo Longo, Federico Pintore, Edoardo Signorini, and Giovanni Tognolini. *A Revision of CROSS Security: Proofs and Attacks for Multi-Round Fiat-Shamir Signatures*. Cryptology ePrint Archive, Paper 2025/127. 2025. URL: https://eprint.iacr.org/2025/127.

[7] Mihir Bellare, Hannah Davis, and Felix Günther. *Separate Your Domains: NIST PQC KEMs, Oracle Cloning and Read-Only Indifferentiability*. Cryptology ePrint Archive, Paper 2020/241. 2020. URL: https://eprint.iacr.org/2020/241.

[8] E. Berlekamp, R. McEliece, and H. van Tilborg. "On the inherent intractability of certain coding problems (Corresp.)" In: *IEEE Transactions on Information Theory* 24.3 (1978), pp. 384–386. DOI: 10.1109/TIT.1978.1055873.

[9] Ward Beullens, Pierre Briaud, and Morten Øygarden. *A Security Analysis of Restricted Syndrome Decoding Problems*. Cryptology ePrint Archive, Paper 2024/611. 2024. URL: https://eprint.iacr.org/2024/611.

[10] Sebastian Bitzer, Jeroen Delvaux, Elena Kirshanova, Sebastian Maaßen, Alexander May, and Antonia Wachter-Zeh. *How to Lose Some Weight - A Practical Template Syndrome Decoding Attack*. Cryptology ePrint Archive, Paper 2024/621. 2024. URL: https://eprint.iacr.org/2024/621.

[11] Niels Olof Bouvin. *Thesis Template - Introduction*. Accessed: 2025-04-01. 2025. URL: https://gitlab.au.dk/au15572/thesis/-/blob/master/Chapters/Chapter01-Introduction.tex.

[12] CROSS Signature team. *seedtree.c: Seed tree implementation in the reference implementation of CROSS*. Accessed: 2025-06-13. 2025. URL: https://github.com/CROSS-signature/CROSS-implementation/blob/main/Reference_Implementation/lib/seedtree.c.

[13] Ming-Shing Chen, Andreas Hülsing, Joost Rijneveld, Simona Samardjiska, and Peter Schwabe. *From 5-pass MQ-based identification to MQ-based signatures*. Cryptology ePrint Archive, Paper 2016/708. 2016. URL: https://eprint.iacr.org/2016/708.

[14] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. "Binary Search Trees." In: *Introduction to Algorithms*. 3rd. MIT Press, 2009. Chap. 12, pp. 286–307. ISBN: 9780262033848.

[15] Ivan Damgård. *An Introduction to Cryptography - Chapter 11*. Aarhus University, Cryptology. 2023.

[16] Ivan Damgård. *On Σ-protocols*. Aarhus University, Cryptographic Protocol Theory. 2023.

[17] Ivan Damgård, Jesper Buus Nielsen, and Sophia Yakoubov. *Commitment Schemes*. Aarhus University, Cryptographic Protocol Theory. 2023.

[18] Decoding Challenge. *Syndrome Decoding Problem*. Accessed: 2025-06-08. URL: https://decodingchallenge.org/syndrome.

[19] Giacomo Fenzi, Jan Gilcher, and Fernando Virdia. *Finding Bugs and Features Using Cryptographically-Informed Functional Testing*. Cryptology ePrint Archive, Paper 2024/1122. 2024. URL: https://eprint.iacr.org/2024/1122.

[20] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. "How to construct random functions." In: *J. ACM* 33 (Aug. 1986), pp. 792–807. DOI: 10.1145/6490.6503.

[21] Google. *Use Cases - The Go Programming Language*. Accessed: 2025-06-06. 2025. URL: https://go.dev/solutions/use-cases.

[22] Google. *crypto/rand: Package rand - Go crypto library*. GoLang Documentation. Accessed: 2025-06-05. 2025. URL: https://pkg.go.dev/crypto/rand#Read.

[23] Matthew Green. *EUF-CMA and SUF-CMA*. A Few Thoughts on Cryptographic Engineering blog. Accessed: 2025-06-13. URL: https://blog.cryptographyengineering.com/euf-cma-and-suf-cma/.

[24] Tomonori Kouya. "Acceleration of Multiple Precision Matrix Multiplication Based on Multi-component Floating-Point Arithmetic Using AVX2." In: *Computational Science and Its Applications – ICCSA 2021*. Springer International Publishing, 2021, 202–217. ISBN: 9783030869762. DOI: 10.1007/978-3-030-86976-2_14. URL: http://dx.doi.org/10.1007/978-3-030-86976-2_14.

[25] Haojun Liu, Xinbo Luo, Hongrui Liu, and Xubo Xia. "Merkle Tree: A Fundamental Component of Blockchains." In: *2021 International Conference on Electronic Information Engineering and Computer Science (EIECS)*. 2021, pp. 556–561. DOI: 10.1109/EIECS53707.2021.9588047.

[26] Shintaro NARISADA, Kazuhide FUKUSHIMA, and Shinsaku KIYOMOTO. "Multiparallel MMT: Faster ISD Algorithm Solving High-Dimensional Syndrome Decoding Problem." In: *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* E106.A.3 (2023), pp. 241–252. DOI: 10.1587/transfun.2022CIP0023.

[27] National Institute of Standards and Technology (NIST) and Morris J. Dworkin. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. FIPS PUB 202, National Institute of Standards and Technology, Gaithersburg, MD. Accessed: 2025-06-08. Aug. 2015. DOI: 10.6028/NIST.FIPS.202. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

[28] National Institute of Standards and Technology. *Announcing the Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication 197. Superseded by FIPS 197-upd1 in 2023. U.S. Department of Commerce, Nov. 2001. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf.

[29] National Institute of Standards and Technology. *NIST Cryptographic Standards and Guidelines Development Process*. Tech. rep. NIST IR 7977. Accessed: 2025-06-06. National Institute of Standards and Technology, 2016. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=920594.

[30] National Institute of Standards and Technology. *PQC Standardization Process: Announcing Four Candidates to be Standardized, Plus Fourth Round Candidates*. Accessed: 2025-06-06. 2022. URL: https://csrc.nist.gov/news/2022/pqc-candidates-to-be-standardized-and-round-4.

[31] National Institute of Standards and Technology. *PQC–Digital Signatures: API Notes*. Accessed: 2025-06-09. 2022. URL: https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/example%20file/api-notes-pqc-dig-sig-page.pdf.

[32] National Institute of Standards and Technology. *PQC–Digital Signatures: Known Answer Tests and Test Vectors*. Accessed: 2025-06-09. 2022. URL: https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/example%20file/kat-pqc-dig-sig-page.pdf.

[33] National Institute of Standards and Technology. *Request for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process*. Accessed: 2025-06-06. 2022. URL: https://csrc.nist.gov/news/2022/request-additional-pqc-digital-signature-schemes.

[34] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. Tech. rep. FIPS 186-5. Accessed: 2025-06-06. U.S. Department of Commerce, 2023. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf.

[35] National Institute of Standards and Technology. *NIST Announces 14 Candidates to Advance to the Second Round of the Additional Digital Signatures for the Post-Quantum Cryptography Standardization Process*. NIST Blog post. Accessed: 2025-06-03. 2024. URL: https://csrc.nist.gov/news/2024/pqc-digital-signature-second-round-announcement.

[36] Ruben Niederhagen. *The MEDS PQC Signature Scheme - How to not become a NIST standard*. Crypto seminar at Aarhus University. Mar. 2025.

[37] OWASP Foundation. *OWASP Top 10:2021*. https://owasp.org/Top10/. Accessed: 2025-06-02. 2021.

[38] Andrew Regenscheid. *Update on the NIST Standardization of Additional Signature Schemes*. Presentation at the Post-Quantum Cryptography Conference, Austin, TX. Jan. 2025. URL: https://pkic.org/events/2025/pqc-conference-austin-us/THU_PLENARY_0900_Update-on-the-NIST-standardization-of-additional-signature-schemes.pdf.

[39] Andrew Regenscheid. *Update on the NIST standardization of additional signature schemes*. Accessed: 2025-06-13. 2025. URL: https://www.youtube.com/watch?v=dDas-NWezUY.

[40] Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. "Dude, is my code constant time?" In: *Proceedings of the Conference on Design, Automation & Test in Europe*. DATE '17. Leuven, BEL: European Design and Automation Association, 2017, 1701–1706.

[41] Prashant Nalini Vasudevan. *The Fiat-Shamir Transformation (CS6230: Lecture 11)*. National University of Singapore, CS6230: Topics in Information Security. 2021. URL: https://www.comp.nus.edu.sg/~prashant/teaching/CS6230/files/notes/lecture11.pdf.

[42] Yolan Romailler. *go-dudect: Toy implementation of Dudect in Go*. GitHub repository, accessed 2025-06-10. 2016. URL: https://github.com/AnomalRoil/go-dudect.

[43] Mark Zhandry. *COS597C: Lecture 4 – Pseudorandom Functions and Signatures*. Tech. rep. Lecture 4. Accessed: 2025-06-08. Princeton University, 2016. URL: https://mzhandry.github.io/courses/2016-Fall-COS597C/ln/ln4.pdf.

[44] Zhide Zhou, Zhilei Ren, Guojun Gao, and He Jiang. "An empirical study of optimization bugs in GCC and LLVM." In: *Journal of Systems and Software* 174 (2021). Accessed: 2025-06-09, p. 110884. ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2020.110884. URL: https://www.sciencedirect.com/science/article/pii/S0164121220302740.

DECLARATION

Both Kristian and Rasmus are responsible for all of the thesis and related code.

Aarhus, June 2025